



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA
CAMPUS DI CESENA

Programmazione di Reti

Laboratorio #7

Andrea Piroddi

Dipartimento di Informatica, Scienza e Ingegneria

SPEED TEST



Come funziona uno Speed Test

Quando si avvia un test di velocità, si verificano più cose.

Innanzitutto, il **client** determina la posizione e il server di test più vicino a voi: questa parte è importante, per evitare che i rallentamenti siano legati alla lunghezza del tragitto e non all'effettiva efficienza della connessione.



Alcune versioni, come *Speedtest.net* di **Ookla**, hanno un'opzione per scegliere il server. Speed Test invia un semplice segnale (ping) al server il quale risponderà. Il test misura quel round trip in millisecondi.

Al termine del ping, inizia il test di download. Il client apre più connessioni al server e tenta di scaricare una piccola porzione di dati. A questo punto, vengono misurate due cose: **il tempo impiegato per acquisire il frammento di dati** e **la quantità di risorse di rete utilizzate**.



Come funziona uno Speed Test

Se il client rileva che è ha risorse disponibili, apre più connessioni al server e scarica più dati.

L'idea generale è di caricare la connessione Internet e vedere quanto può trasportare simultaneamente.

Immaginate il servizio Internet come un'autostrada con un limite di velocità.

Aprire connessioni aggiuntive è come aggiungere più corsie all'autostrada.

Il limite di velocità non è cambiato, ma più auto possono attraversare lo stesso spazio a una velocità maggiore;

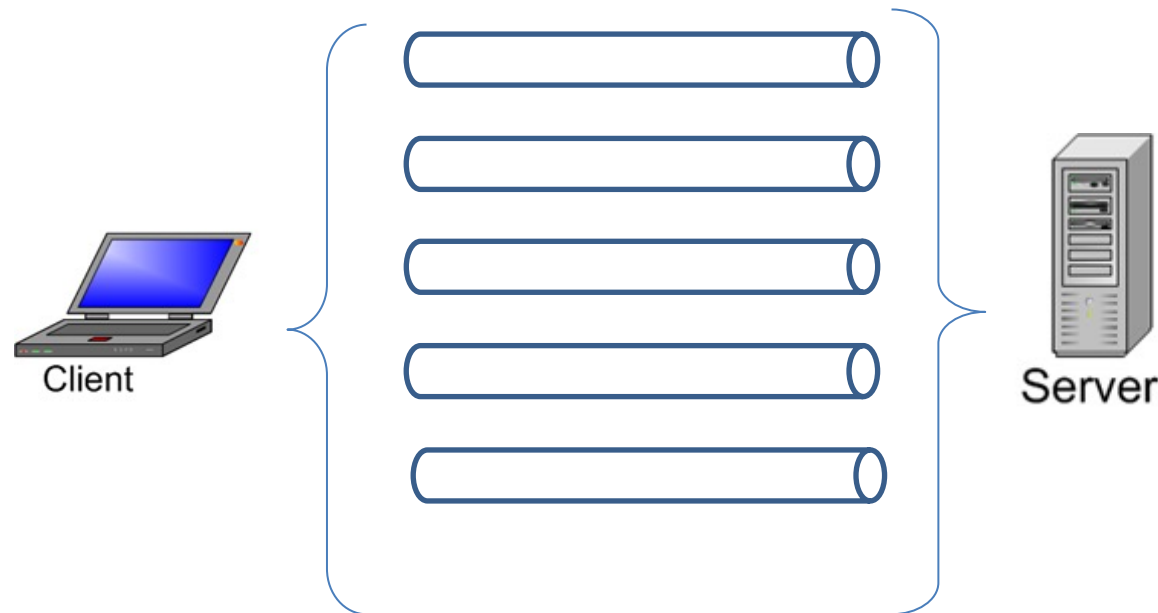
quindi, la 50^a auto arriverà prima usando un'autostrada a quattro corsie rispetto a quella su una a due corsie.



Come funziona uno Speed Test

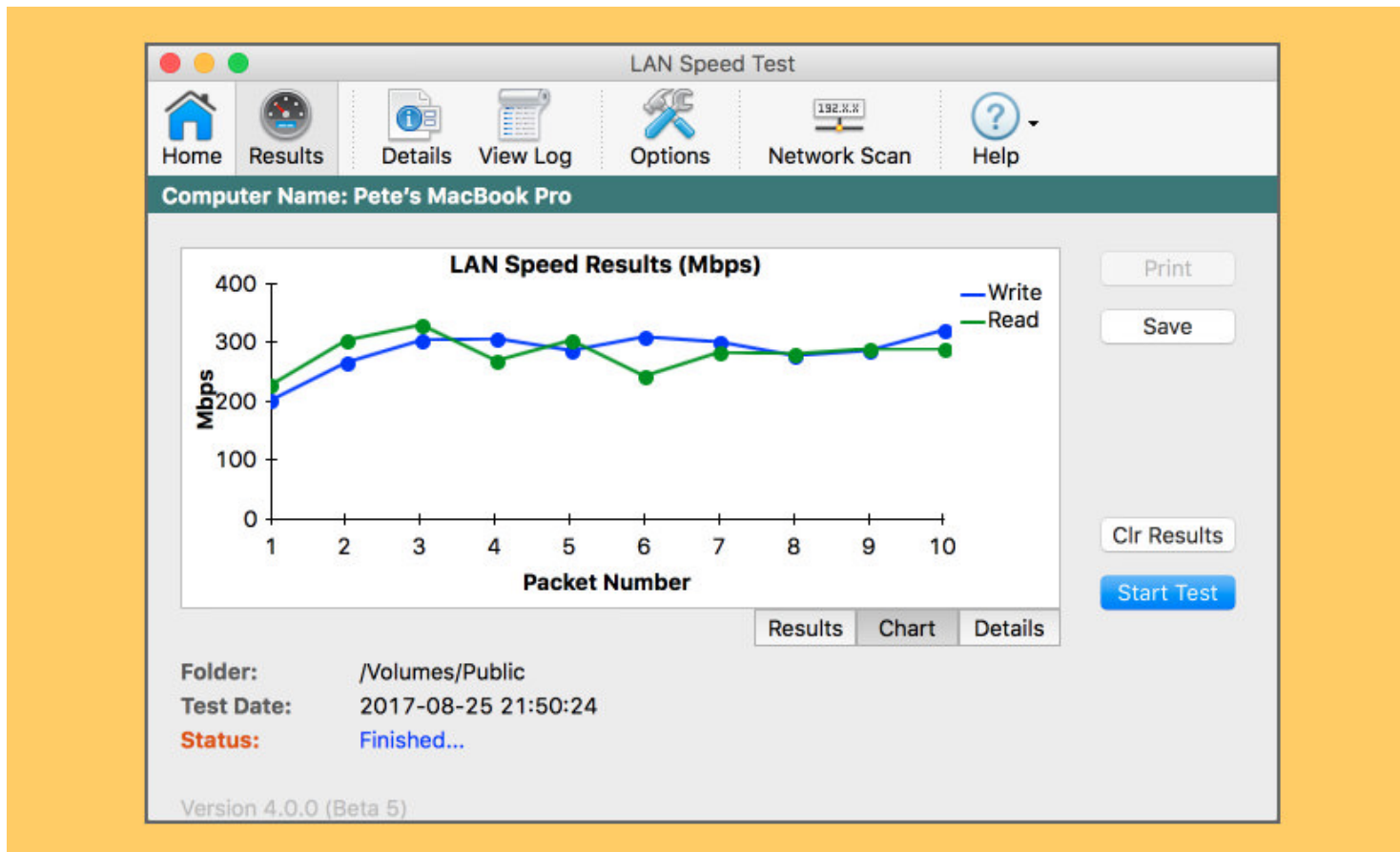
Una volta che il **client** ha accertato che dispone delle connessioni per testare il servizio Internet, scarica ulteriori blocchi di dati, misura la quantità scaricata nel tempo assegnato e presenta **una velocità di download**.

Il test di upload è essenzialmente lo stesso processo del test di download ma al contrario. Invece di estrarre i dati dal server al PC, il client carica i dati dal PC al server.



Calcolo Speed Test

Nell'esempio che segue, effettueremo solo una connessione come esempio, ovviamente è possibile moltiplicare quanto vedremo su più connessioni in parallelo per avere una idea più precisa dell'effettivo Throughput disponibile.



Speed Test in Python



Speed Test in Python

```
import sys, time
from socket import *
```

```
MY_PORT = 50000 + 42
BUFSIZE = 1024
```



```
def main():
    if len(sys.argv) < 2:
        usage()
    if sys.argv[1] == '-s':
        server()
    elif sys.argv[1] == '-c':
        client()
    else:
        usage()

def usage():
    sys.stdout = sys.stderr
    print ('Usage:      (on host_A) throughput -s [port]')
    print ('and then: (on host_B) throughput -c count host_A [port]')
    sys.exit(2)
```



Definiamo la porta

Definiamo la dimensione del Buffer

Creiamo due funzioni una per la gestione degli input da riga di comando finalizzata alla esecuzione dello script in modalità server o client e l'altra per la gestione di errori in fase di input

Si ricordi che sys.argv è un vettore di stringhe → sys.argv=[]



Speed Test Lato Server



Speed Test Lato Server

```
def server():
    if len(sys.argv) > 2:
        port = eval(sys.argv[2])
    else:
        port = MY_PORT
    s = socket(AF_INET, SOCK_STREAM)
    s.bind(('', port))
    s.listen(1)
    print ('Server ready...')
    while 1:
        conn, (host, remoteport) = s.accept()
        while 1:
            data = conn.recv(BUFSIZE)
            if not data:
                break
            del data
            conn.send(('OK\n').encode())
            conn.close()
        print ('Done with', host, 'port', remoteport)
```

- Lato server dobbiamo ovviamente definire la porta di ascolto
 - La funzione ***eval()*** serve a trasformare il valore di ingresso (considerato stringa) in un valore intero
 - Se non assegno una porta in fase di esecuzione del comando, lo script prende quella dichiarata nella variabile **MY_PORT**
- Se non arrivano dati sul socket lo script termina, altrimenti cicla assegnando in ogni ciclo alla variabile data il contenuto del buffer, che poi svuota prima di rimettersi in ascolto.



Speed Test Lato Client

```
def client():
    if len(sys.argv) < 4:
        usage()
    count = int(eval(sys.argv[2]))
    host = sys.argv[3]
    if len(sys.argv) > 4:
        port = eval(sys.argv[4])
    else:
        port = MY_PORT
    testdata = 'x' * (BUFSIZE-1) + '\n'
    t1 = time.time()
    s = socket(AF_INET, SOCK_STREAM)
    t2 = time.time()
    s.connect((host, port))
    t3 = time.time()
    i = 0
    while i < count:
        i = i+1
        s.send(testdata.encode())
    s.shutdown(1) # Send EOF
    t4 = time.time()
    data = s.recv(BUFSIZE)
    t5 = time.time()
    print (data)
    print ('Raw timers:', t1, t2, t3, t4, t5)
    print ('Intervals:', t2-t1, t3-t2, t4-t3, t5-t4)
    print ('Total:', t5-t1)
    print ('Throughput:', round((BUFSIZE*count*0.001) / (t5-t1), 3), 'K/sec.')
```

- Se la lunghezza del vettore sys.argv è inferiore a 4 significa che mancano informazioni in ingresso ossia il numero di pacchetti da inviare e l'indirizzo e porta del server
- Se la porta viene indicata verrà selezionata quella, altrimenti verrà aperta la connessione verso quella definita nello script
- Quindi il client «riempie» il buffer di 1024 byte con una stringa di tutte x e un «a capo» finale (dummy burst) → xxxxxx....+'\n'

- E Comincia ad inviare i dummy burst verso il server. Quando il contatore ha terminato, invia l'EOF.



Calcolo del THROUGHPUT

All'istante t_5 il Client ha terminato l'invio di tutti burst ed esegue il calcolo seguente:
Il tempo totale di Trasmissione è:

$$T = t_5 - t_1$$

Quantità di **bytes** trasmessi complessivamente:

$$Bytes_{tot} = n^o pacchetti\ inviati \times 1024 bytes = COUNT \times 1024 bytes$$

$$Throughput\ in\ \frac{Kbyte}{sec} = \frac{Bytes_{tot}}{T} = \frac{COUNT \times 1024}{T} \times 0.001 \left[\frac{kB}{s} \right]$$



Calcolo del RTT (Round Trip Time)

Per calcolare il Round Trip Time possiamo prendere l'istante di invio dell' EOF (t_4) da parte del client e l'istante (t_5) in cui il client riceve il messaggio OK dal server

$$RTT = t_5 - t_4 [s]$$

```
Linux ubuntunet2008 2.6.35-32-generic-pae #68-Ubuntu SMP Tue Mar 27 18:04:42 UTC 2012 i686 GNU/Linux
Ubuntu 10.10

Welcome to Ubuntu!
 * Documentation:  https://help.ubuntu.com/

New release 'natty' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Fri Apr 24 15:51:20 2020 from 192.168.1.141
apirodd@ubuntunet2008:~$ python BandwidthClient_Server.py -c 10000 192.168.1.141 11000
OK

Raw timers: 1587743148.59 1587743148.59 1587743148.59 1587743155.7 1587743155.74
Intervals: 4.50611114502e-05 0.00301384925842 7.10320401192 0.0448861122131
Total: 7.1511490345
Throughput: 1431.938 K/sec.
```



Risultati

Nel nostro esempio abbiamo ottenuto un **Throughput** di quasi

1.5MB/s (ossia circa 12Mb/s)

E un

$$RTT = 1587743155.74 - 1587743155.7 = 0.0448 \text{ [sec]} \cong \mathbf{45ms}$$



Wireshark per tracciare il transito dei pacchetti

The image shows a Wireshark packet capture window titled 'speed_test.pcapng'. The packet list pane shows several packets. A green box highlights packets 84 through 97, which are SMB and SSH traffic. A text box with the text 'Pacchetti di instaurazione del tunnel' points to this highlighted area. The packet details pane shows the structure of the selected packet (Frame 84), including Ethernet II, Internet Protocol Version 4, Transmission Control Protocol, and SMB (Server Message Block Protocol). The packet bytes pane shows the raw data of the selected packet.

No.	Time	Source	Destination	Protocol	Length	Info
82	4.608255	192.168.1.144	224.0.0.251	MDNS	87	Standard query 0x0000 PTR _spotify-connect._tcp.local "OU" question
83	4.610104	192.168.1.144	239.255.255.250	SSDP	167	M-SEARCH * HTTP/1.1
84	5.160947	192.168.1.141	192.168.1.117	SMB	107	Echo Request
85	5.172713	192.168.1.117	192.168.1.141	SMB	107	Echo Response
86	5.212528	192.168.1.141	192.168.1.117	TCP	54	55391 → 445 [ACK] Seq=54 Ack=54 Win=4559 Len=0
87	5.610180	192.168.1.141	192.168.1.117	SSH	94	Client: Encrypted packet (len=40)
88	5.625852	192.168.1.117	192.168.1.141	SSH	94	Server: Encrypted packet (len=40)
89	5.652843	192.168.1.117	192.168.1.141	TCP	74	60520 → 11000 [SYN] Seq=0 Win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=1164635391 TSecr=0 WS=64
90	5.653087	192.168.1.141	192.168.1.117	TCP	66	11000 → 60520 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM=1
91	5.654331	192.168.1.117	192.168.1.141	TCP	60	60520 → 11000 [ACK] Seq=1 Ack=1 Win=5888 Len=0
92	5.654914	192.168.1.117	192.168.1.141	TCP	1078	60520 → 11000 [PSH, ACK] Seq=1 Ack=1 Win=5888 Len=1024
93	5.655583	192.168.1.117	192.168.1.141	TCP	1514	60520 → 11000 [ACK] Seq=1025 Ack=1 Win=5888 Len=1460
94	5.655585	192.168.1.117	192.168.1.141	TCP	1514	60520 → 11000 [ACK] Seq=2485 Ack=1 Win=5888 Len=1460
95	5.655709	192.168.1.141	192.168.1.117	TCP	54	11000 → 60520 [ACK] Seq=1 Ack=3945 Win=65536 Len=0
96	5.657365	192.168.1.117	192.168.1.141	TCP	1514	60520 → 11000 [ACK] Seq=3945 Ack=1 Win=5888 Len=1460
97	5.657475	192.168.1.141	192.168.1.117	TCP	54	11000 → 60520 [ACK] Seq=1 Ack=5405 Win=65536 Len=0

> Frame 84: 107 bytes on wire (856 bits), 107 bytes captured (856 bits) on interface \Device\NPF_{A2E3F05E-2636-424E-BC62-F71E17BA3406}, id 0
> Ethernet II, Src: IntelCor_53:0e:5d (b4:d5:bd:53:0e:5d), Dst: HewlettP_18:87:50 (1c:c1:de:18:87:50)
> Internet Protocol Version 4, Src: 192.168.1.141, Dst: 192.168.1.117
> Transmission Control Protocol, Src Port: 55391, Dst Port: 445, Seq: 1, Ack: 1, Len: 53
> NetBIOS Session Service
> SMB (Server Message Block Protocol)

```
0000 1c c1 de 18 87 50 b4 d5 bd 53 0e 5d 08 00 45 00 .....P...S.]..E-
0010 00 5d b8 18 40 00 08 06 be 2f c0 a8 01 8d c0 a8 .].@.../.....
0020 01 75 d8 5f 01 bd 20 b8 c3 6e 9b 54 28 26 50 18 .u....n-T(&P-
0030 11 cf ba 35 00 00 00 00 00 31 ff 53 4d 42 2b 00 ...5....1SMB+
0040 00 00 00 18 43 c0 00 00 00 00 00 00 00 00 00 ....C.....
0050 00 00 ff ff ff fe 00 00 fe ff 01 01 00 0c 00 4a .....]
0060 6c 4a 6d 49 68 43 6c 42 73 72 00 1jmIhClB sr-
```

Come si vede nella figura al lato la connessione tra il client

192.168.1.117

(speedtester) e il server

192.168.1.141

(speedtester)

avviene tramite protocollo SSH

Si ricordi che nel nostro speed test è il client ad inviare i pacchetti verso il server



Wireshark per tracciare il transito dei pacchetti

speed_test.pcapng

File Modifica Visualizza Vai Cattura Analizza Statistiche Telefonica Wireless Strumenti Aiuto

Applica un filtro di visualizzazione ... <Ctrl-F>

No.	Time	Source	Destination	Protocol	Length	Info
88	5.625852	192.168.1.117	192.168.1.141	SSH	94	Server: Encrypted packet (len=40)
89	5.652843	192.168.1.117	192.168.1.141	TCP	74	60520 → 11000 [SYN] Seq=0 Win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=1164635391 TSecr=0 WS=64
90	5.653087	192.168.1.141	192.168.1.117	TCP	66	11000 → 60520 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM=1
91	5.654331	192.168.1.117	192.168.1.141	TCP	60	60520 → 11000 [ACK] Seq=1 Ack=1 Win=5888 Len=0
92	5.654914	192.168.1.117	192.168.1.141	TCP	1078	60520 → 11000 [PSH, ACK] Seq=1 Ack=1 Win=5888 Len=1024
93	5.655583	192.168.1.117	192.168.1.141	TCP	1514	60520 → 11000 [ACK] Seq=1025 Ack=1 Win=5888 Len=1460
94	5.655585	192.168.1.117	192.168.1.141	TCP	1514	60520 → 11000 [ACK] Seq=2485 Ack=1 Win=5888 Len=1460
95	5.655709	192.168.1.141	192.168.1.117	TCP	54	11000 → 60520 [ACK] Seq=1 Ack=3945 Win=65536 Len=0
96	5.657365	192.168.1.117	192.168.1.141	TCP	1514	60520 → 11000 [ACK] Seq=3945 Ack=1 Win=5888 Len=1460
97	5.657475	192.168.1.141	192.168.1.117	TCP	54	11000 → 60520 [ACK] Seq=1 Ack=5405 Win=65536 Len=0
98	5.658389	192.168.1.117	192.168.1.141	TCP	1514	60520 → 11000 [ACK] Seq=5405 Ack=1 Win=5888 Len=1460
99	5.658391	192.168.1.117	192.168.1.141	TCP	1514	60520 → 11000 [ACK] Seq=6865 Ack=1 Win=5888 Len=1460
100	5.658398	192.168.1.117	192.168.1.141	TCP	1514	60520 → 11000 [ACK] Seq=8325 Ack=1 Win=5888 Len=1460
101	5.658520	192.168.1.141	192.168.1.117	TCP	54	11000 → 60520 [ACK] Seq=1 Ack=9785 Win=65536 Len=0
102	5.659913	192.168.1.117	192.168.1.141	TCP	1514	60520 → 11000 [ACK] Seq=9785 Ack=1 Win=5888 Len=1460
103	5.660019	192.168.1.141	192.168.1.117	TCP	54	11000 → 60520 [ACK] Seq=1 Ack=11245 Win=65536 Len=0

> Frame 100: 1514 bytes on wire (12112 bits), 1514 bytes captured (12112 bits) on interface \Device\NPF_{A2E3F05E-2636-424E-BC62-F71E17BA3406}, id 0

> Ethernet II, Src: HewlettP_18:87:50 (1c:c1:de:18:87:50), Dst: IntelCor_53:0e:5d (b4:d5:bd:53:0e:5d)

> Internet Protocol Version 4, Src: 192.168.1.117, Dst: 192.168.1.141

> Transmission Control Protocol, Src Port: 60520, Dst Port: 11000, Seq: 8325, Ack: 1, Len: 1460

> Data (1460 bytes)

0000 b4 d5 bd 53 0e 5d 1c c1 de 18 87 50 08 00 45 00 ...S-]-...P-E-

0010 05 dc 59 8f 40 00 40 06 57 3a c0 a8 01 75 c0 a8 ...Y.@-W:...u-

0020 01 8d ec 68 2a f8 c2 6a d5 fd 21 3b 84 89 50 10 ...h-j-!-P-

0030 00 5c 48 ca 00 00 78 78 78 78 78 78 78 78 78 ...H-..xx xxxxxxxx

0040 78 78 78 78 78 78 78 78 78 78 78 78 78 78 78 xxxxxxxx xxxxxxxx

0050 78 78 78 78 78 78 78 78 78 78 78 78 78 78 78 xxxxxxxx xxxxxxxx

0060 78 78 78 78 78 78 78 78 78 78 78 78 78 78 78 xxxxxxxx xxxxxxxx

0070 78 78 78 78 78 78 78 78 78 78 78 78 78 78 78 xxxxxxxx xxxxxxxx

0080 78 78 78 78 78 78 78 78 78 78 78 78 78 78 78 xxxxxxxx xxxxxxxx

0090 78 78 78 78 78 78 78 78 78 78 78 78 78 78 78 xxxxxxxx xxxxxxxx

Si vede il contenuto dei pacchetti inviati dal client.

Nella cartella del Laboratorio è presente il file di tracciamento di Wireshark



ARP



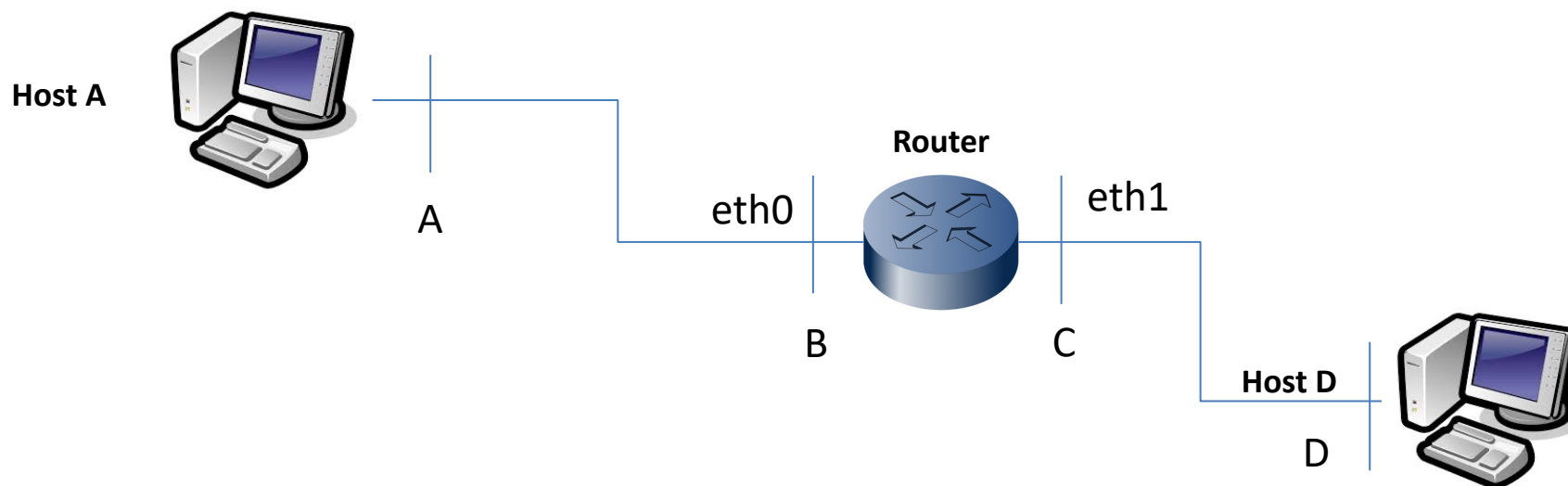
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA
CAMPUS DI CESENA

Esercizio 4



Esercizio 4 – Tabella ARP

Si consideri la rete in figura dove le interfacce sono identificate con lettere maiuscole



Si indichino con IP-x e MAC-x, con $x=[A,B,C,D]$, gli indirizzi IP e ethernet delle interfacce



Esercizio 4 – Tabella ARP

Si supponga che le ARP table di A, di B e del router siano vuote

L'host A deve inviare un pacchetto IP verso l'indirizzo IP-D.

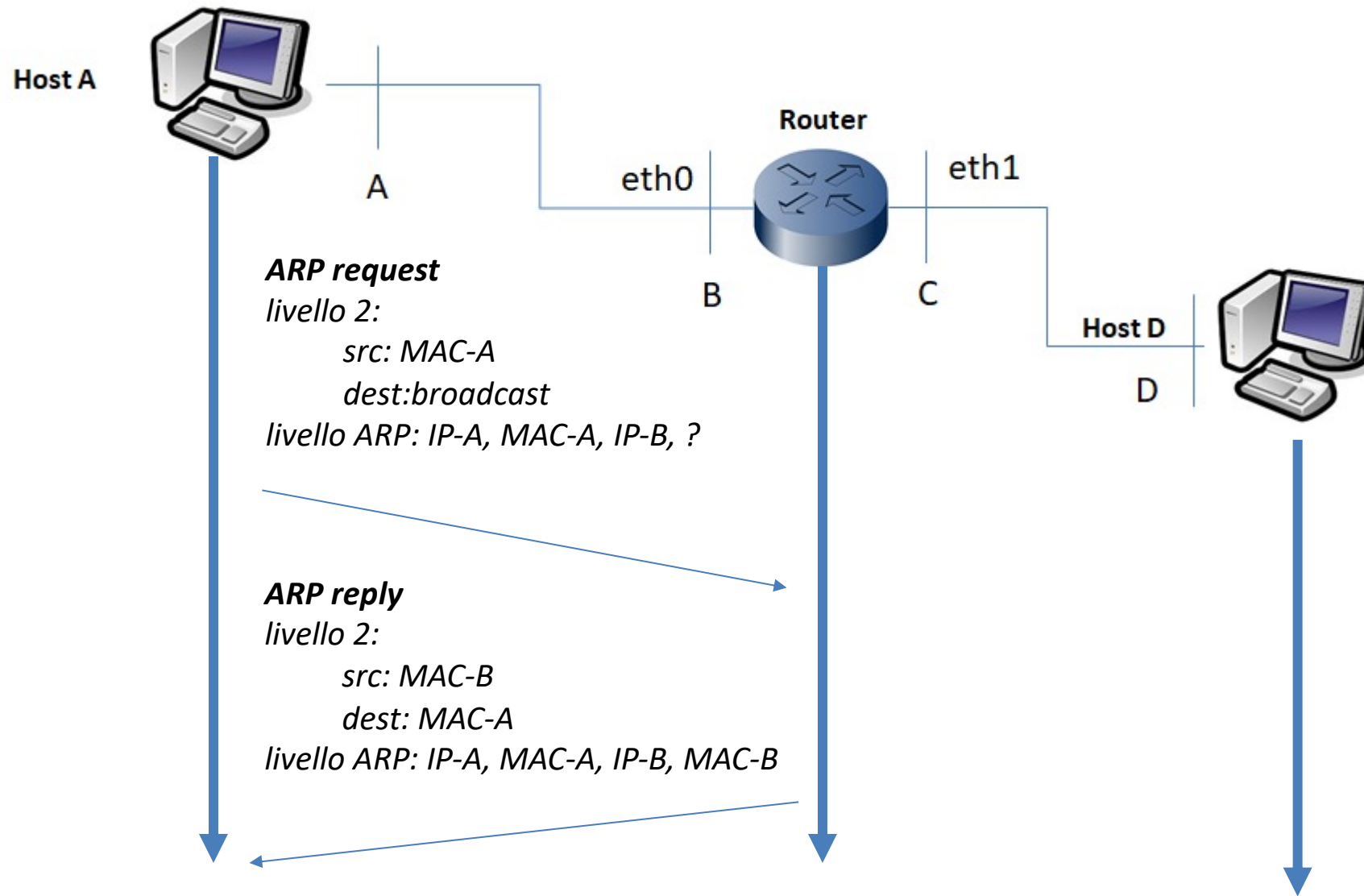
Ricevuto il pacchetto l'host D deve inviare un pacchetto di risposta verso A.

Quesiti

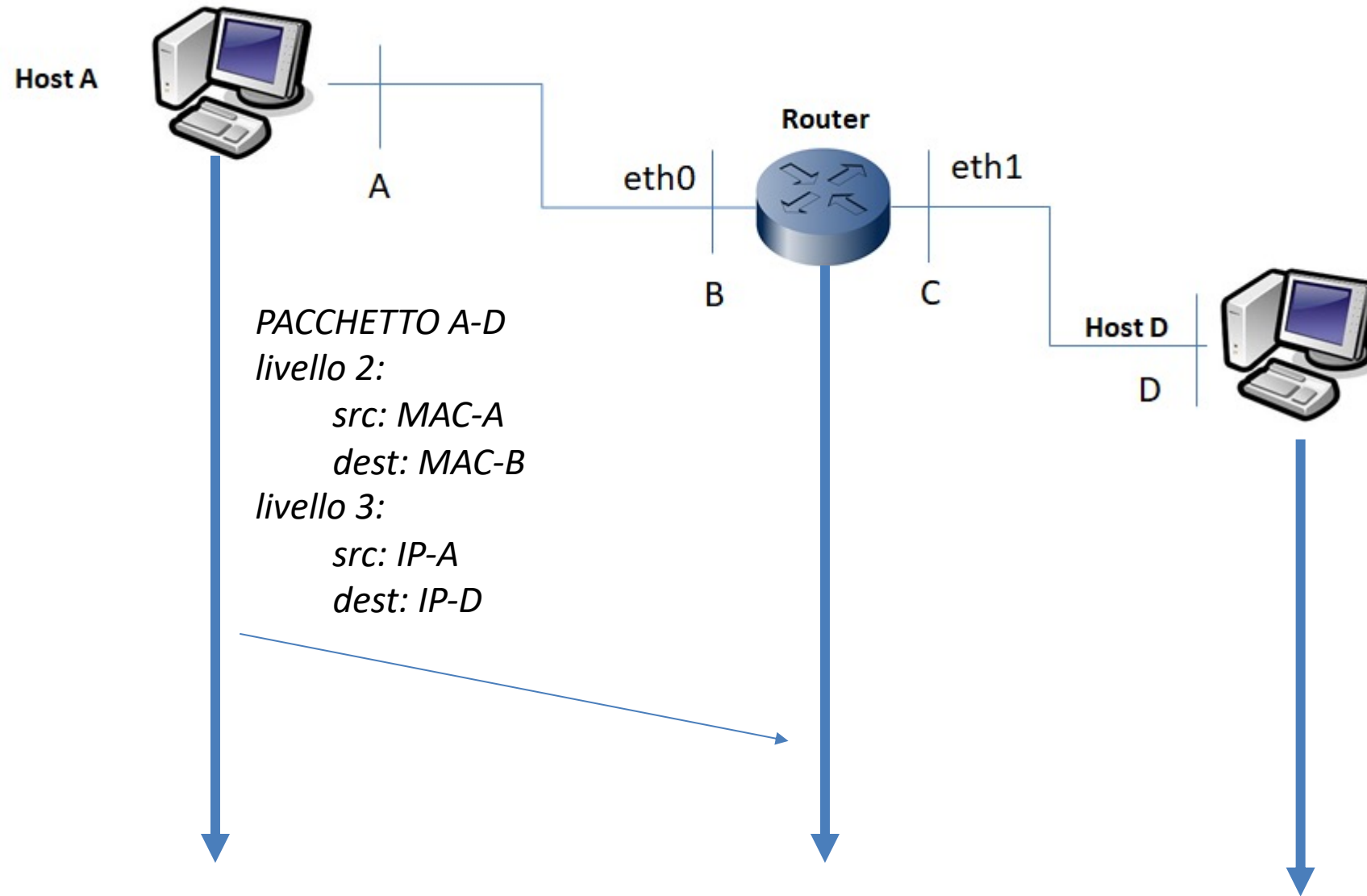
1. si indichino graficamente i pacchetti che vengono trasmessi e per ciascuno di essi (su ognuna delle reti ethernet attraversate) gli indirizzi contenuti nelle PDU di livello 2 (ethernet) e 3 (IP o ARP)



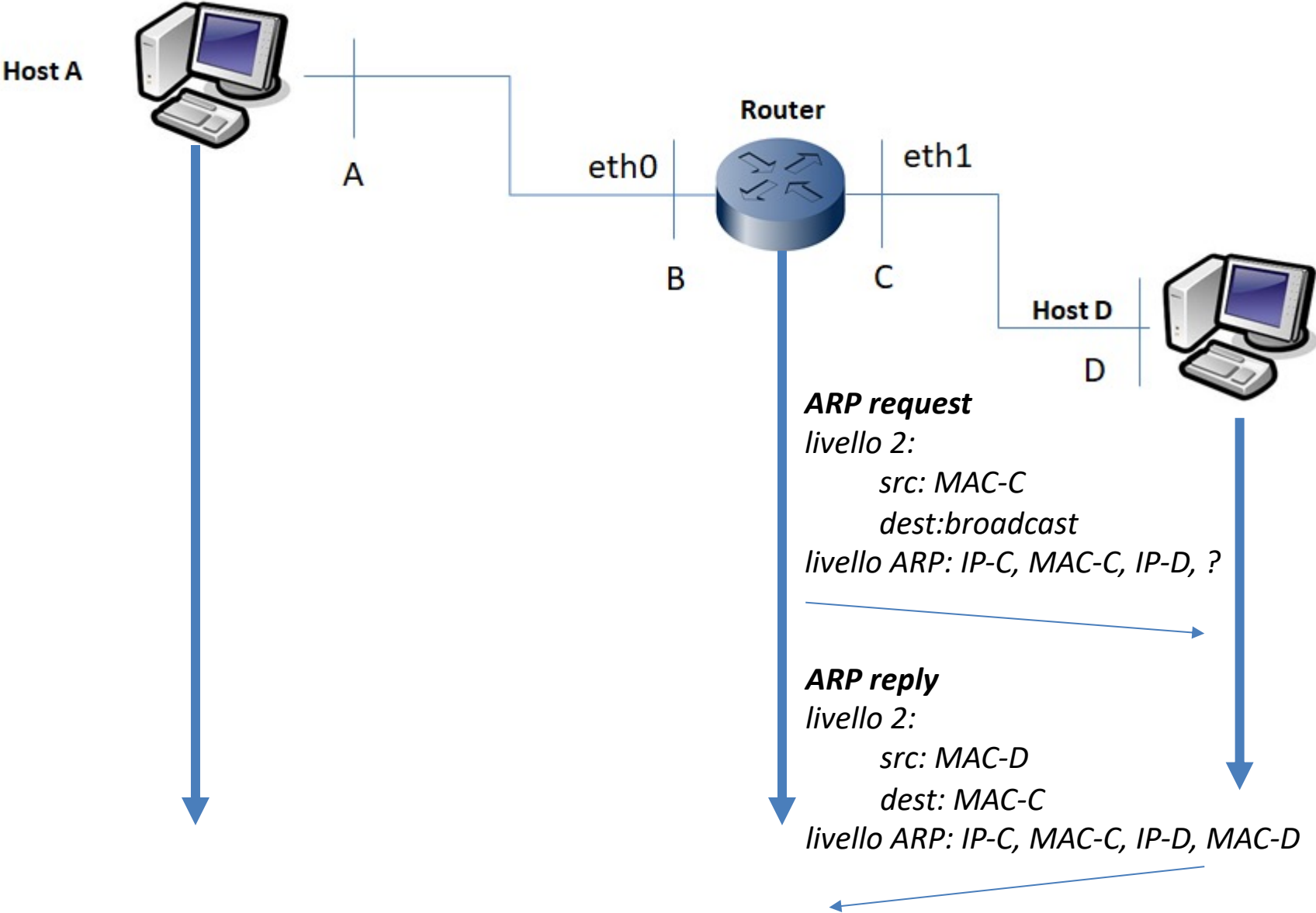
Esercizio 4 – Tabella ARP - soluzione



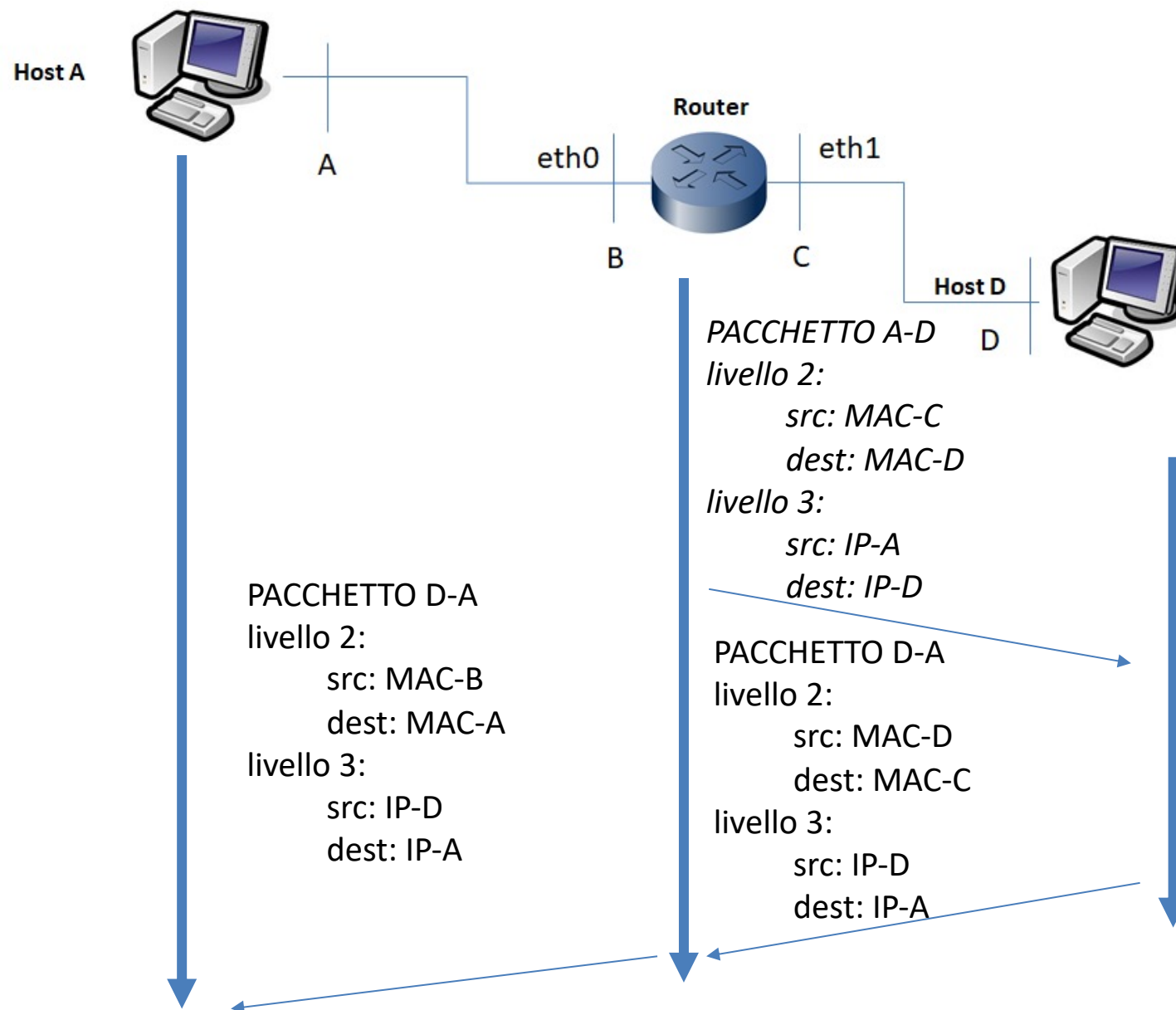
Esercizio 4 – Tabella ARP - soluzione



Esercizio 4 – Tabella ARP - soluzione



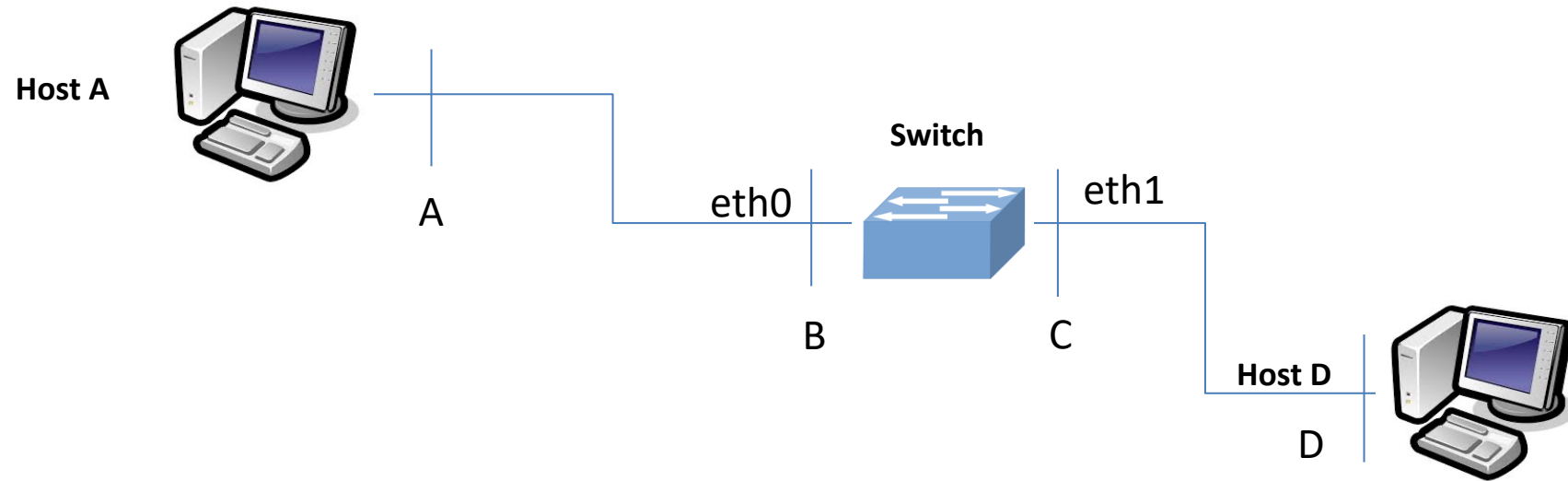
Esercizio 4 – Tabella ARP - soluzione



Esercizio 5



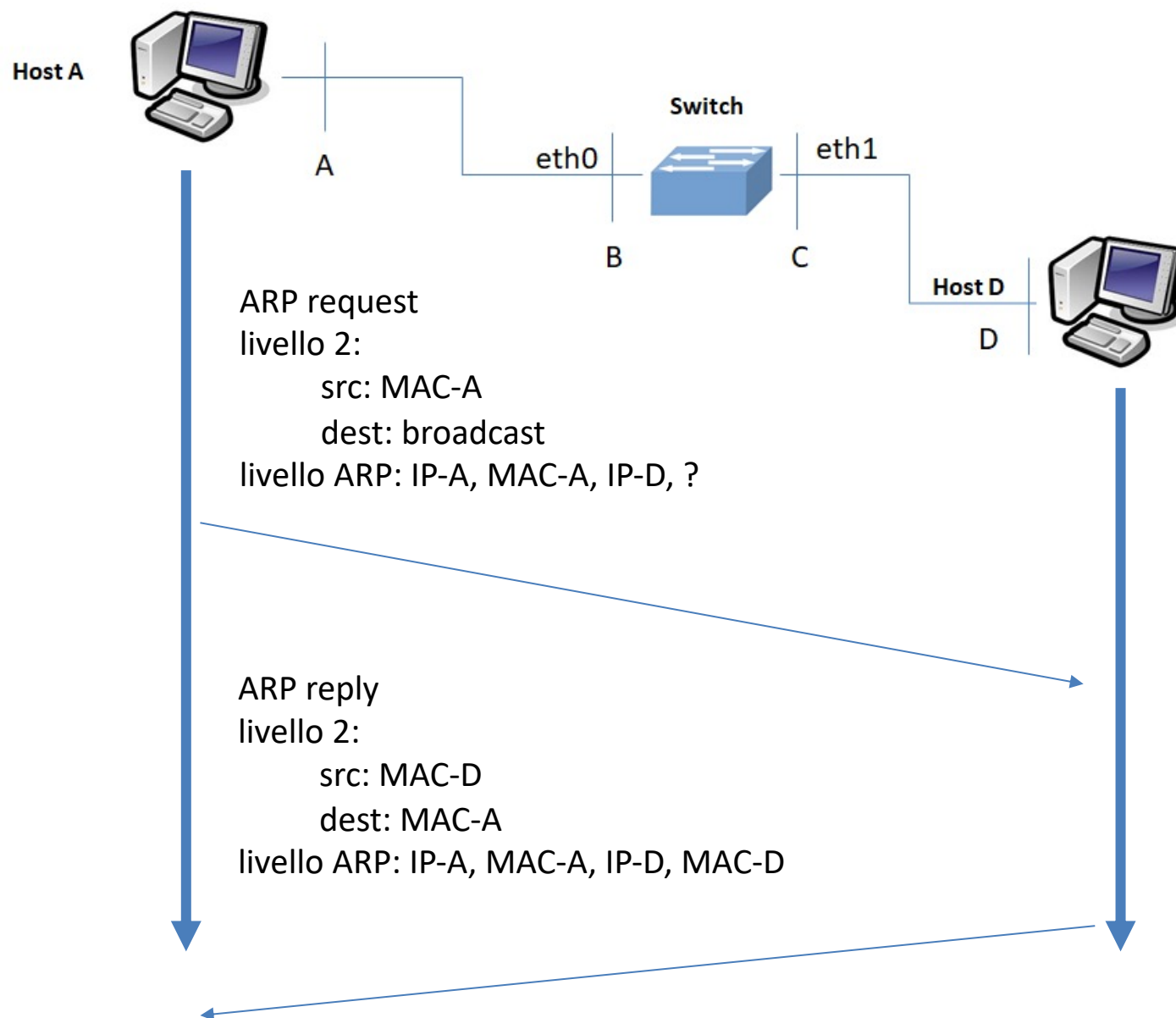
Esercizio 5 – Tabella ARP



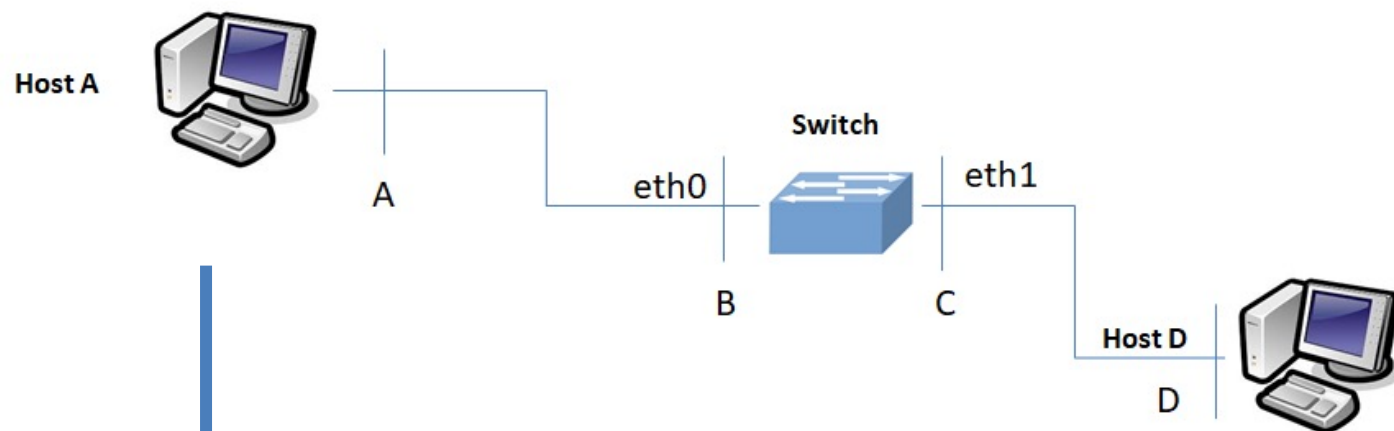
Lo stesso nel caso in cui il nodo al centro sia un bridge (layer 2 switch)



Esercizio 5 – Tabella ARP - soluzione



Esercizio 5 – Tabella ARP - soluzione



PACCHETTO A-D

livello 2:

src: MAC-A

dest: MAC-D

livello 3:

src: IP-A

dest: IP-D

PACCHETTO D-A

livello 2:

src: MAC-D

dest: MAC-A

livello 3:

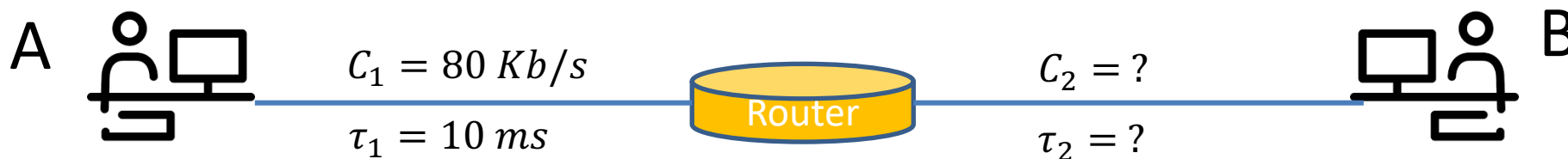
src: IP-D

dest: IP-A



Esercizio 2

Si consideri il collegamento in figura



QUESITO

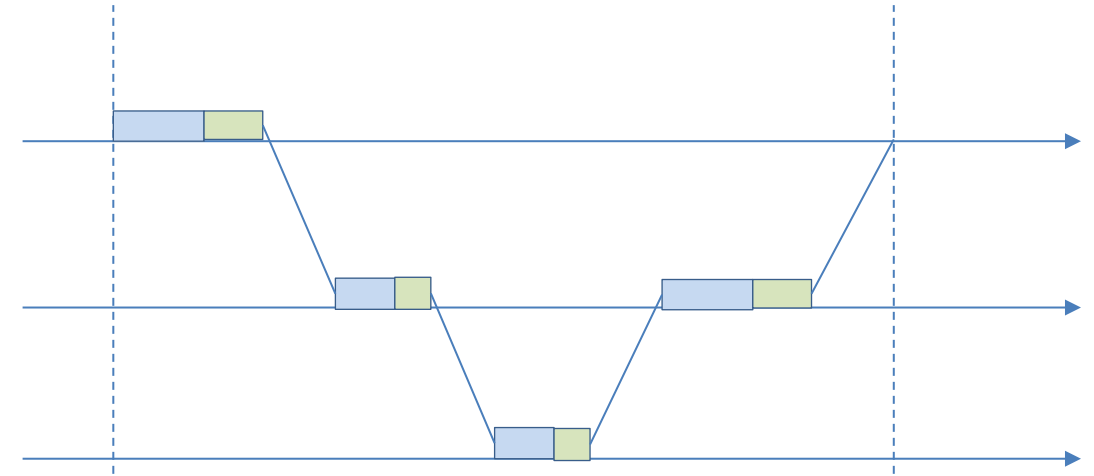
- A vuole conoscere la capacità e il ritardo di propagazione del link 2 e allo scopo invia a B 2 messaggi di echo: M_1 di lunghezza $L_1 = 1000 \text{ [byte]}$, ed M_2 di lunghezza $L_2 = 1500 \text{ [byte]}$; per ognuno di essi misura un Round-Trip-Time (RTT) pari a 780 [ms] e 1130 [ms] rispettivamente. Nella risposta, B utilizza messaggi con le stesse lunghezze. Calcolare C_2 e τ_2 nell'ipotesi che le lunghezze degli header siano trascurabili.



Soluzione Esercizio 2 – Quesito 1

$$RTT_1 = 2 \cdot \left(\frac{L_1}{c_1} + \frac{L_1}{c_2} + \tau_1 + \tau_2 \right) = 780 [ms]$$

$$RTT_2 = 2 \cdot \left(\frac{L_2}{c_1} + \frac{L_2}{c_2} + \tau_1 + \tau_2 \right) = 1130 [ms]$$



Da cui si ricava che

$$\frac{780 [ms]}{2} - \left(\frac{L_1}{c_1} + \frac{L_1}{c_2} \right) = (\tau_1 + \tau_2) \text{ sostituendo nella seconda equazione}$$

$$\frac{1130 [ms]}{2} = \left(\frac{L_2}{c_1} + \frac{L_2}{c_2} \right) + \frac{780 [ms]}{2} - \left(\frac{L_1}{c_1} + \frac{L_1}{c_2} \right) = \left(\frac{12000 [bit]}{80 \cdot 10^3 [\frac{bit}{s}]} + \frac{12000 [bit]}{c_2} \right) + \frac{780 [ms]}{2} - \left(\frac{8000 [bit]}{80 \cdot 10^3 [\frac{bit}{s}]} + \frac{8000 [bit]}{c_2} \right)$$

$$175 [ms] = 50 [ms] + \frac{4000 [bit]}{c_2}$$

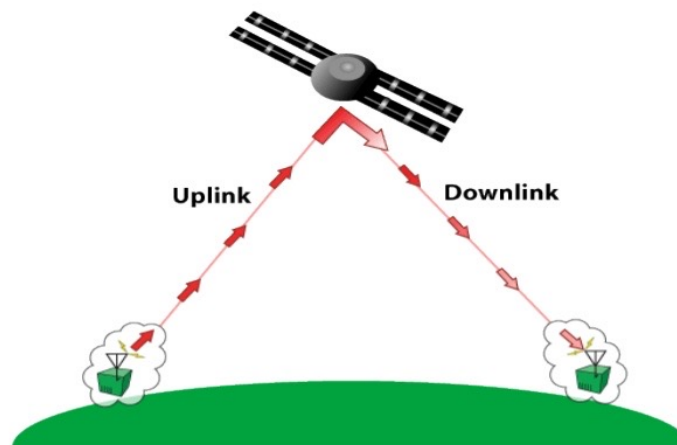
$$c_2 = \frac{4000 [bit]}{125 [ms]} = \frac{4000 [bit]}{125 \cdot 10^{-3} [s]} = \mathbf{32 [\frac{Kb}{s}]}$$

$$\tau_2 = \frac{780 [ms]}{2} - \left(\frac{L_1}{c_1} + \frac{L_1}{c_2} \right) - \tau_1 = 390 [ms] - 100 [ms] - 250 [ms] - 10 [ms] = \mathbf{30 [ms]}$$



Esercizio 3

Si consideri un canale via satellite della capacità di 1 [Mb/s] e che il tempo di propagazione attraverso il satellite geostazionario richieda 250 [ms] . Si suppongano gli ACK trascurabili.

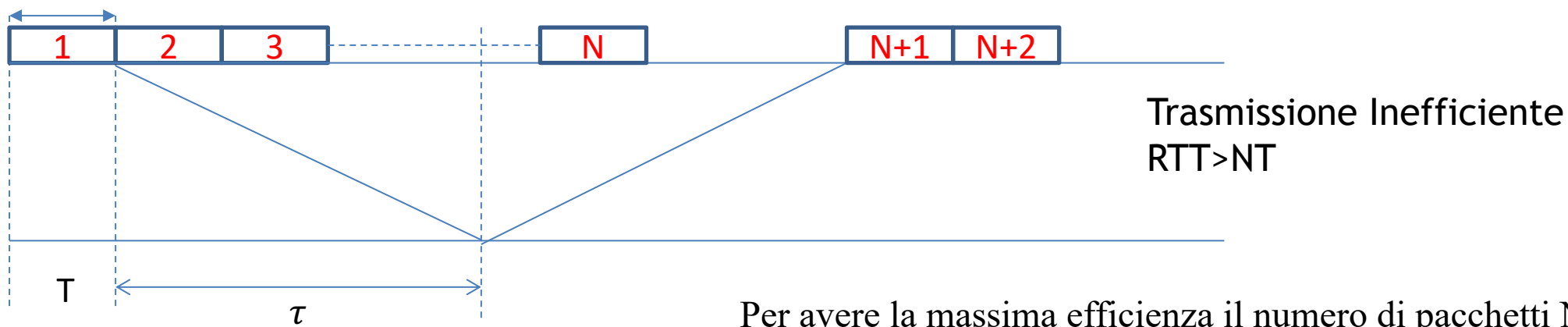


- Si chiede di dimensionare la minima finestra di trasmissione di un protocollo Go-BACK-N (con timeout) in modo che sia consentita la massima efficienza temporale del canale quando vengano trasmesse trame di 2000 [bit] in assenza di errori.
- Si calcoli poi la massima efficienza trasmissiva che si avrebbe nel caso in cui il meccanismo ARQ fosse di tipo STOP and WAIT

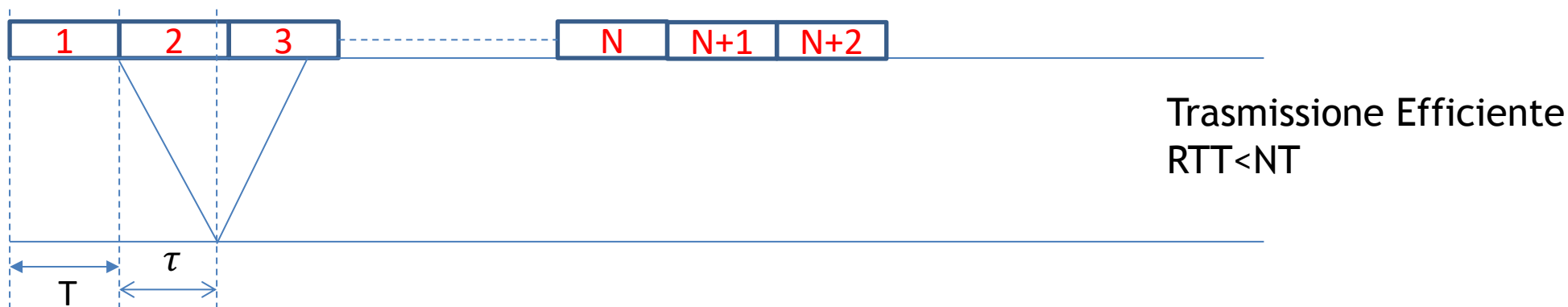


Soluzione Esercizio 3 – Quesito 1

L'efficienza del meccanismo Go-BACK-N dipende dal rapporto tra RTT e lunghezza della finestra.



Per avere la massima efficienza il numero di pacchetti N nella finestra deve essere tale che il loro tempo di trasmissione copra il tempo di andata e ritorno del primo pacchetto



Soluzione Esercizio 3 – Quesito 1

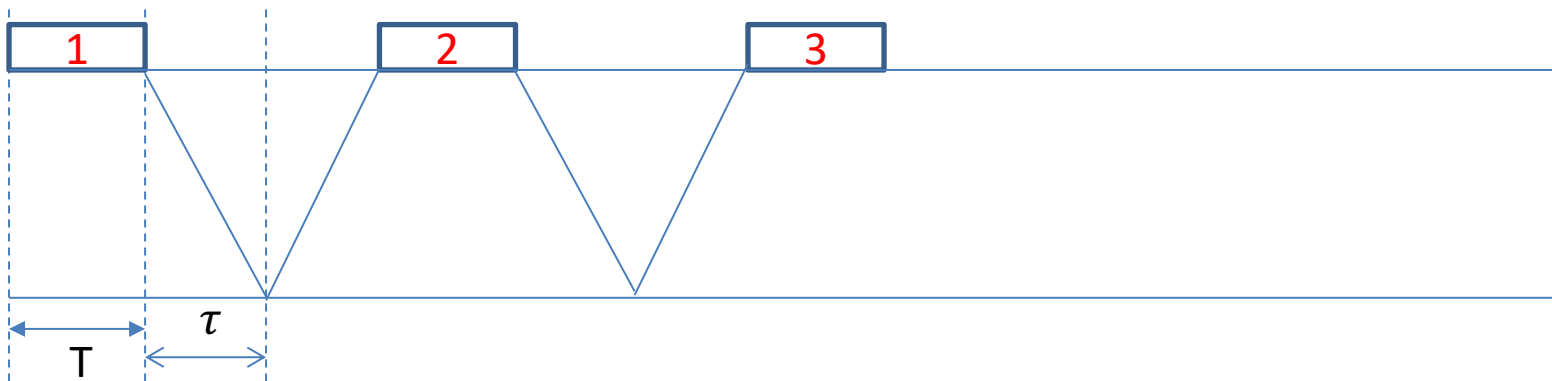
Detti:

- $T = 2 \text{ [ms]} = 2000 \text{ [bit]} / 1 \text{ [Mb/s]}$, il tempo di trasmissione di un pacchetto
- $t = 250 \text{ [ms]}$, il tempo di propagazione

allora deve essere: $NT \geq T + 2t \rightarrow N > 1 + \frac{2t}{T} = 1 + 2 \cdot \frac{250}{2} = 251$

Per finestre $N \geq 251$, la trasmissione risulta continua, dunque l'efficienza del meccanismo è 1.

Nel caso di Stop & Wait abbiamo:



L'efficienza dello Stop & Wait si calcola considerando che il protocollo trasmette 1 pacchetto (durata $T=2 \text{ [ms]}$)

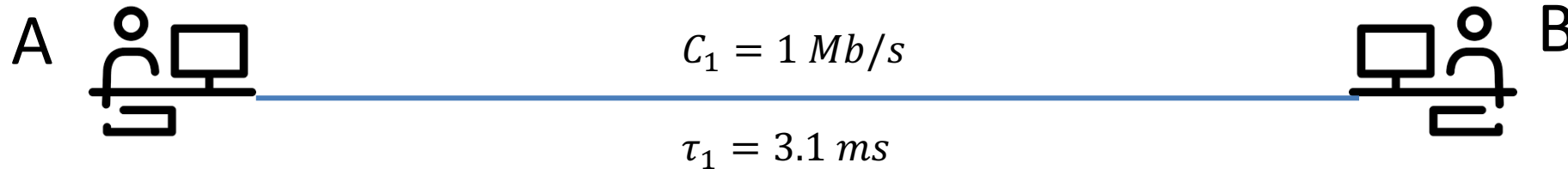
ogni $T+2t$, dunque l'efficienza: $\eta = \frac{T}{(T+2t)} = \frac{1}{251}$



Esercizio 4

Si consideri il collegamento in figura.

L'Host A deve trasferire tramite canale TCP una sequenza di 100 segmenti di lunghezza massima.



QUESITO

Si calcoli il tempo necessario, supponendo che:

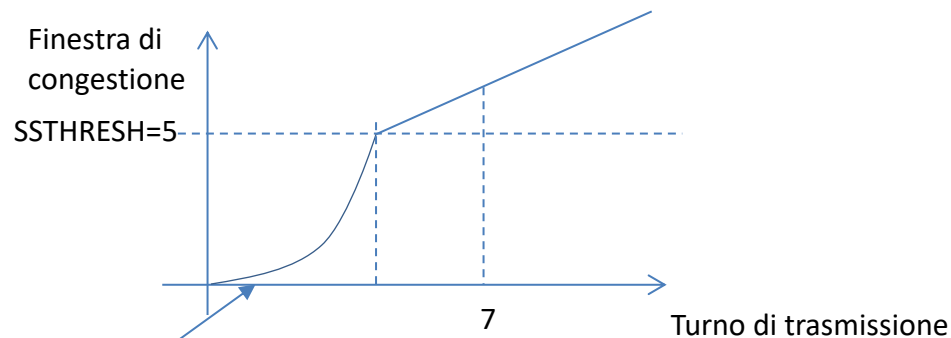
- $MSS = 1000 \text{ [bit]}$
- Lunghezza degli Header di tutti i livelli trascurabile
- La connessione viene aperta da A e la lunghezza dei segmenti di apertura della connessione è trascurabile
- La lunghezza degli ACK è trascurabile
- $SSTHRESH$ è pari a 5 MSS



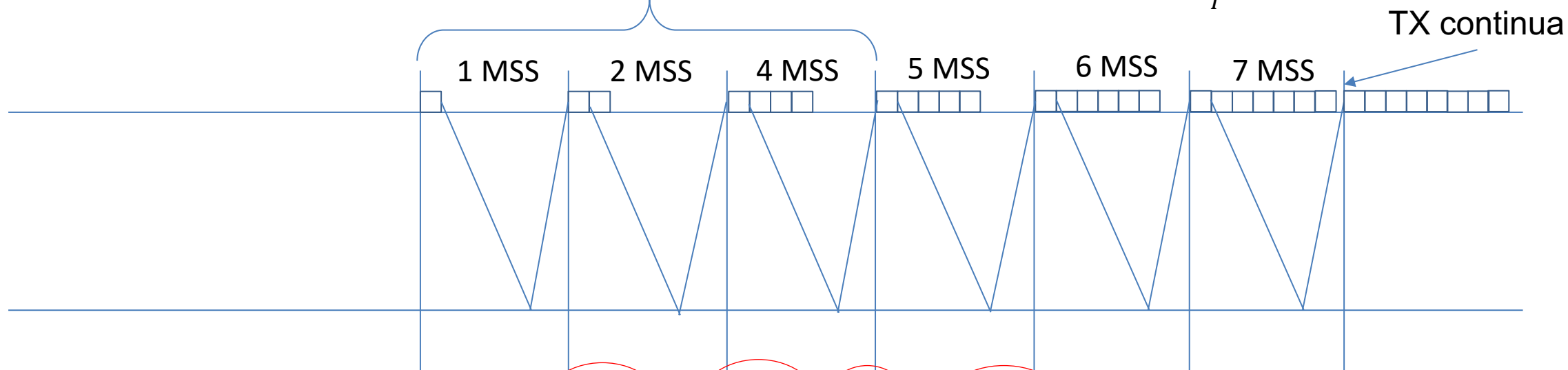
Soluzione Esercizio 4 – Quesito 1

$$T = \frac{1000 [bit]}{1 [\frac{Mb}{s}]} = 1 [ms]$$

$$RTT = 2 \cdot \tau_1 + T = 6.2 [ms] + 1 [ms]$$



La trasmissione è discontinua fino a che $W \cdot T < RTT$, cioè fino a che $W < \frac{RTT}{T} \approx 8$



$$T_{transfer} = 6.2 [ms] + 6 RTT + 75 \cdot T + 6.2 [ms] = 130.6 [ms]$$

Set up
(abbiamo
ipotizzato che
la lunghezza
dei segmenti
di apertura sia
trascurabile)

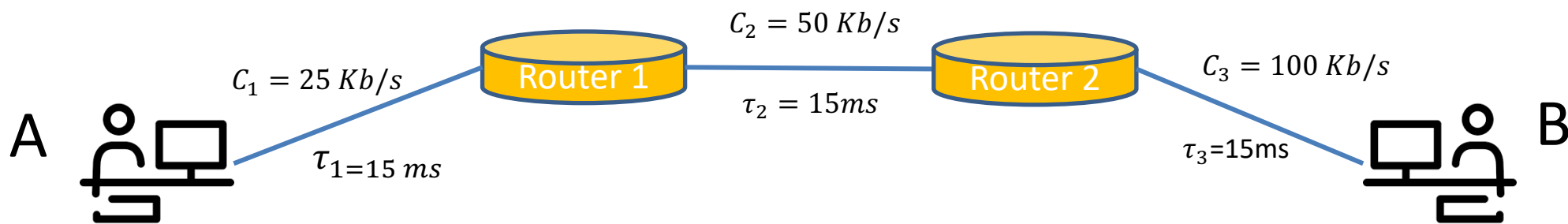
Primi 25 MSS
(ossia i primi 6 turni
 $1+2+4+5+6+7=25$)

Ultimi 75 MSS

Ack ultimo MSS



Esercizio 5



All'istante 0 viene attivata una connessione TCP tra l'host A e l'host B.

QUESITI:

- Si calcoli l'istante di tempo oltre il quale la trasmissione sul link 1 risulta continua, supponendo
 - ☐ *header* trascurabili
 - ☐ *link* bidirezionali e simmetrici
 - ☐ $\text{RCWND} = 4000 \text{ [byte]}$ e $\text{SSTHRESH} = 400 \text{ [byte]}$
 - ☐ dimensione segmenti $\text{MSS} = 200 \text{ [byte]}$
 - ☐ dimensione ACK =dimensione segmenti per apertura della connessione = 20 [byte]
 - ☐ Connessione aperta dal terminale A
- Quanto tempo occorre per trasferire un file da 2 [kbyte] (dall'istante di trasmissione del primo segmento all'istante di ricezione dell'ACK dell'ultimo segmento)?



Soluzione Esercizio 5 – Quesito 1

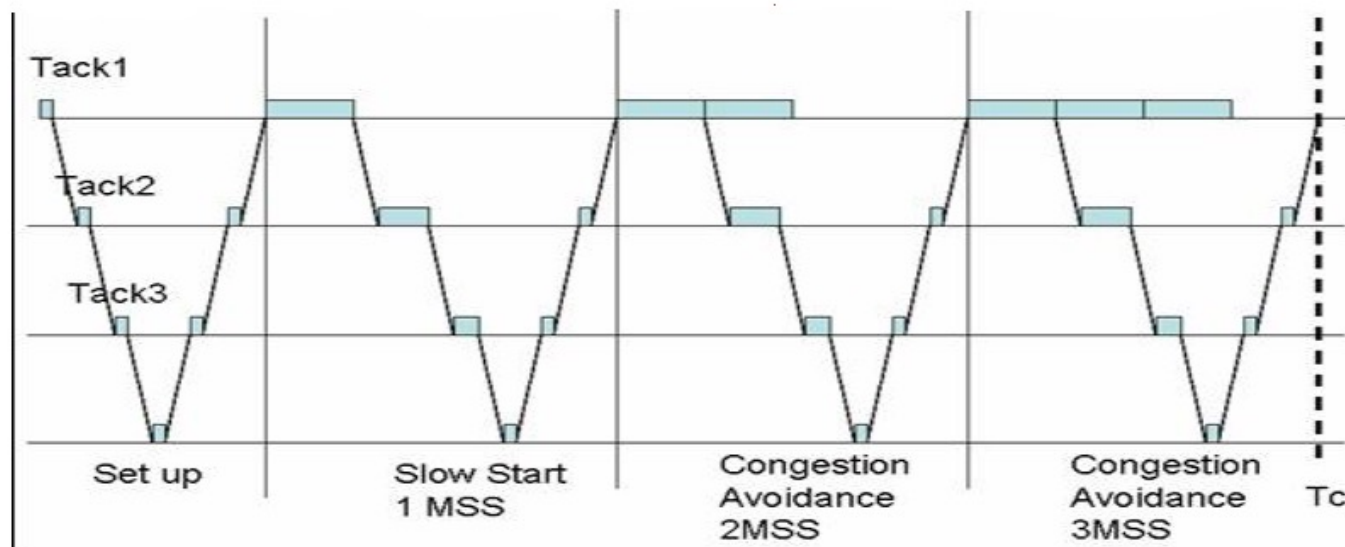
Calcoliamo un po' di valori:

- $T_1 = \frac{200 \cdot 8 \text{ [bit]}}{25 \left[\frac{\text{kbit}}{\text{s}}\right]} = 64 \text{ ms}$

- $T_2 = \frac{1}{2} T_1 = 32 \text{ ms}$

- $T_3 = \frac{1}{2} T_2 = 16 \text{ ms}$

- $RTT = T_1 + T_2 + T_3 + 2 \cdot (\tau_1 + \tau_2 + \tau_3) + (T_{ack_1} + T_{ack_2} + T_{ack_3}) = 64 \text{ [ms]} + 32 \text{ [ms]} + 16 \text{ [ms]} + 2 \cdot (45 \text{ [ms]}) + (6.4 \text{ [ms]} + 3.2 \text{ [ms]} + 1.6 \text{ [ms]}) = 213.2 \text{ ms}$



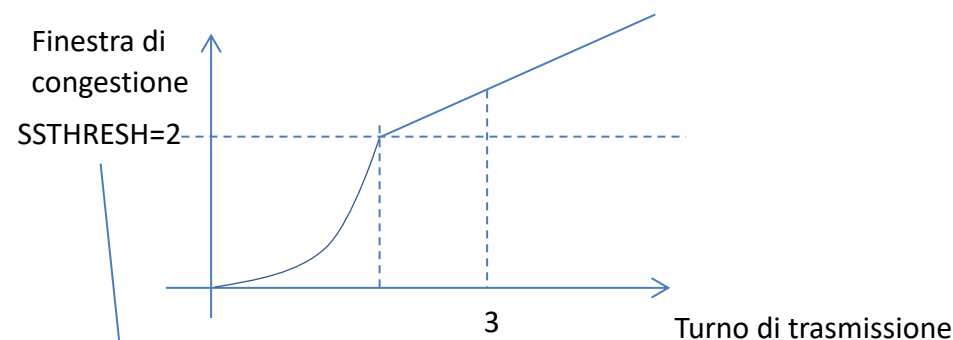
Soluzione Esercizio 5 – Quesito 1

- $T_{setup} = 2 \cdot (T_{ack_1} + T_{ack_2} + T_{ack_3}) + 2 \cdot (\tau_1 + \tau_2 + \tau_3) = 2 \cdot (6.4 [ms] + 3.2 [ms] + 1.6 [ms]) + 2 \cdot (45 [ms]) = 112.4 ms$
- Il link 1 è il collo di bottiglia, trasmissione è continua sul link 1 quando:
 $WT_1 > RTT$
- Quindi $W > \frac{RTT}{T_1} = \frac{213.2 ms}{64 ms} = 3.3$
- L'istante in cui la trasmissione diventa continua è

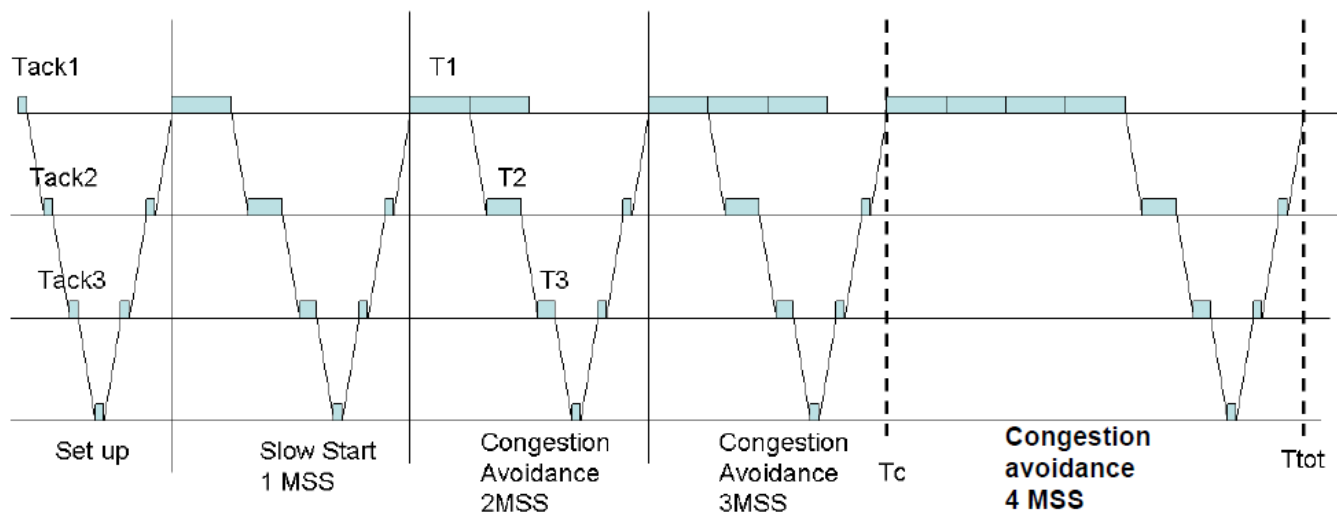
$$T_c = T_{setup} + 3 \cdot RTT = 112.4 [ms] + 3 \cdot 213.2 [ms] = 752 [ms]$$



Soluzione Esercizio 5 – Quesito 2



- Il file da trasferire è di 2 [kbyte], equivalenti a 10 MSS.
- Il tempo per trasferire 10 MSS è: $T_c = T_{setup} + 4 \cdot RTT + 3 \cdot T_1 = 1.15 [s]$



ossia i primi 4 turni ($W > 3.3$),
rimangono poi 3 MSS per completare
l'invio



Esercizio 6

- Una connessione TCP è usata per trasmettere un file da 39.5 [kbyte] utilizzando i seguenti parametri:
 - MSS=500 [byte]
 - RTT = 500 [ms]
 - timeout T1 = 2*RTT.
- Si assuma che le condizioni iniziali delle finestre siano:
 - RCWND = 12 [kbyte]
 - Ssthresh = 8 [kbyte]
 - CWND = 500 [byte]
- E che inoltre:
 - si verifichi un errore sulla connessione all'istante 3 [s] (tutti i segmenti in trasmissione vengano persi)
 - al tempo 4,5 [s] il ricevitore segnali RCWND = 2 [kbyte]
- Si tracci l'andamento nel tempo di:
 - CWND
 - Ssthresh
 - RCWND
- Si calcoli il tempo di trasmissione del file



Soluzione Esercizio 6 – Quesito 1

Conviene ragionare in numero di segmenti trasmessi

$$\text{Dimensione File (in MSS)} = \frac{39.5 \text{ [Kbyte]}}{500 \text{ [byte]}} = 79 \text{ MSS}$$

Dobbiamo trovare il tempo necessario per trasferire 79 MSS

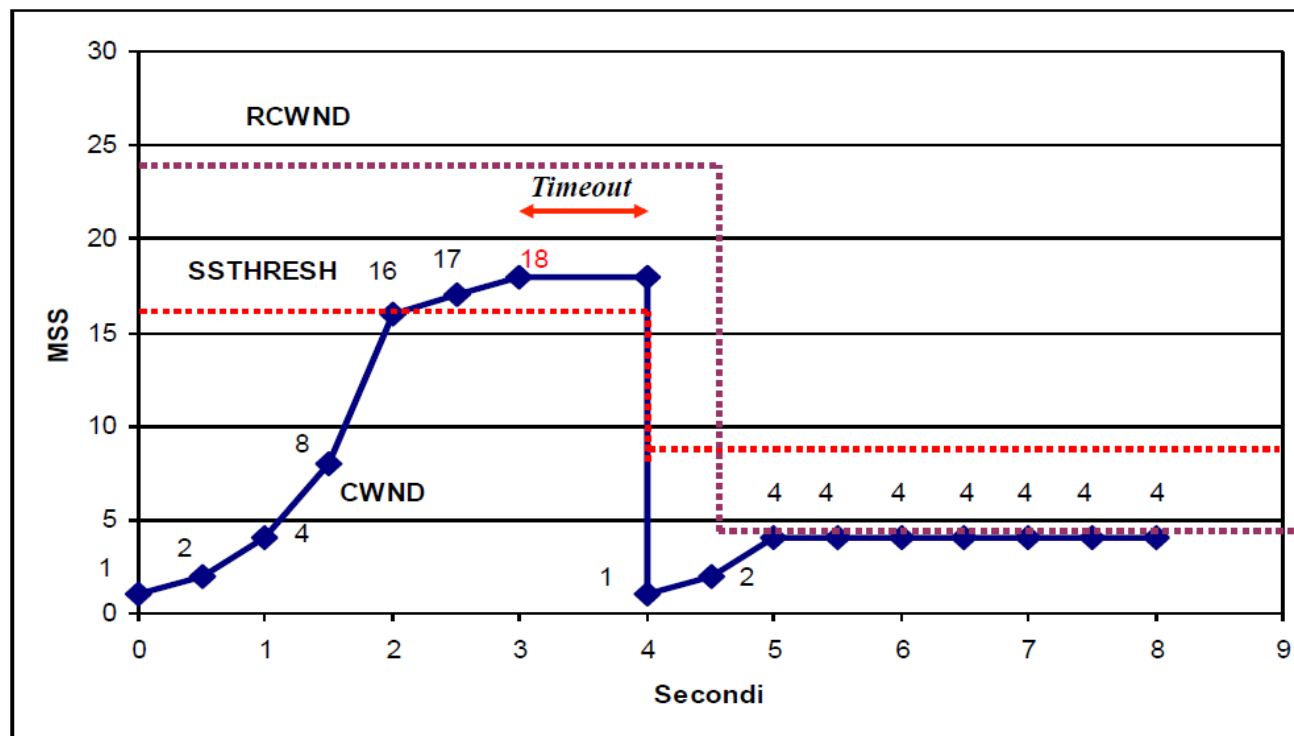
$$\text{RCWND} = \frac{12 \text{ [kbyte]}}{500 \text{ [byte]}} = 24 \text{ MSS}$$

$$\text{SSTHRESH} = \frac{8 \text{ [kbyte]}}{500 \text{ [byte]}} = 16 \text{ MSS}$$

$$\text{Timeout} = 1 \text{ s}$$

Tempo di trasferimento del file:

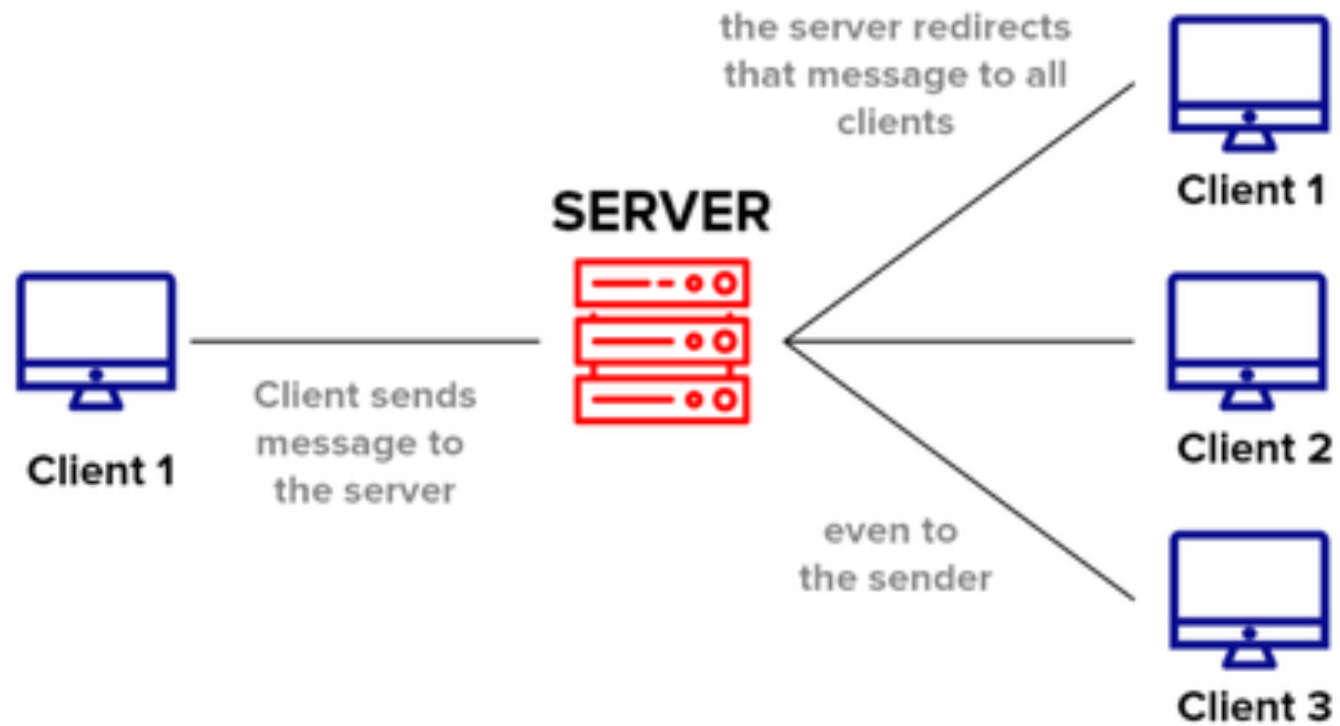
$$T = 8.5 \text{ [s]}$$



CHAT CLIENT SERVER



CHAT client - server



Lato SERVER



Servizio CHAT – Lato Server

```
clients = {}  
indirizzi = {}  
  
HOST = ''  
PORT = 53000  
BUFSIZ = 1024  
ADDR = (HOST, PORT)  
  
SERVER = socket(AF_INET, SOCK_STREAM)  
SERVER.bind(ADDR)
```

Definiamo gli elementi iniziali:

- Due dizionari per la registrazione dei client in ingresso e dei relativi nomi associati agli indirizzi
- La porta e l'interfaccia del server
- E il bind



Servizio CHAT – Lato Server

```
#!/usr/bin/env python3
"""Script Python per la realizzazione di un Server multithread
per connessioni CHAT asincrone.
Corso di Programmazione di Reti - Università di Bologna"""

from socket import AF_INET, socket, SOCK_STREAM
from threading import Thread
```

Per questo scopo useremo i socket TCP.

Quindi importiamo dal modulo socket gli elementi di cui abbiamo bisogno.

In linea di principio potremmo usare anche i socket UDP.

Chiaramente, TCP si adatta meglio al nostro intento rispetto ai socket UDP.



```
{<socket.socket fd=752, family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0, laddr=('127.0.0.1', 53000), raddr=('127.0.0.1', 62825)>: ('127.0.0.1', 62825)}
```

Servizio CHAT – Lato Server

```
""" La funzione che segue accetta le connessioni dei client in entrata."""
def accetta_connessioni_in_entrata():
    while True:
        client, client_address = SERVER.accept()
        print("%s:%s si è collegato." % client_address)
        #al client che si connette per la prima volta fornisce alcune indicazioni di utilizzo
        client.send(bytes("Salve! Digita il tuo Nome seguito dal tasto Invio!", "utf8"))
        # ci serviamo di un dizionario per registrare i client
        indirizzi[client] = client_address
        #diamo inizio all'attività del Thread - uno per ciascun client
        Thread(target=gestisce_client, args=(client,)).start()
```

- Definiamo una funzione che gestisca le connessioni in entrata.
- Quindi quando viene invocata, essa si mette in ascolto sul socket, e appena arriva una richiesta di connessione da un client, il server gli restituisce un messaggio di saluto accompagnato da alcune indicazioni sull'utilizzo della chat.
- Prende il dizionario «indirizzi» e lo aggiorna con il dato relativo al nuovo client
- Infine attiva il **Thread()** (*start()*) verso questo client andando ad invocare la funzione «*gestisce_client*»

ATTENZIONE: *start()* deve essere chiamato al massimo una volta per oggetto thread.

Questo metodo genererà un *RuntimeError* se chiamato più di una volta sullo stesso oggetto thread.



Servizio CHAT – Lato Server

```
{<socket.socket fd=404, family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0, laddr=('127.0.0.1', 33000),  
raddr=('127.0.0.1', 62495)>: 'Name1',  
<socket.socket fd=708, family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0, laddr=('127.0.0.1', 33000),  
raddr=('127.0.0.1', 62505)>: 'Name2'}
```

```
"""La funzione seguente gestisce la connessione di un singolo client."""  
def gestice_client(client): # Prende il socket del client come argomento della funzione.  
    nome = client.recv(BUFSIZ).decode("utf8")  
    #da il benvenuto al client e gli indica come fare per uscire dalla chat quando ha terminato  
    benvenuto = 'Benvenuto %s! Se vuoi lasciare la Chat, scrivi {quit} per uscire.' % nome  
    client.send(bytes(benvenuto, "utf8"))  
    msg = "%s si è unito all chat!" % nome  
    #messaggio in broadcast con cui vengono avvisati tutti i client connessi che l'utente x è entrato  
    broadcast(bytes(msg, "utf8"))  
    #aggiorna il dizionario clients creato all'inizio  
    clients[client] = nome  
  
#si mette in ascolto del thread del singolo client e ne gestisce l'invio dei messaggi o l'uscita dalla Chat  
while True:  
    msg = client.recv(BUFSIZ)  
    if msg != bytes("{quit}", "utf8"):  
        broadcast(msg, nome+": ")  
    else:  
        client.send(bytes("{quit}", "utf8"))  
        client.close()  
        del clients[client]  
        broadcast(bytes("%s ha abbandonato la Chat." % nome, "utf8"))  
        break
```

Definiamo una funzione che si occuperà di gestire il **Thread()** per ciascun client che entra nella Chat. All'interno della funzione in oggetto verrà chiamata la funzione ***broadcast*** per inviare i messaggi a tutti gli utenti.



Servizio CHAT – Lato Server

```
""" La funzione, che segue, invia un messaggio in broadcast a tutti i client."""  
def broadcast(msg, prefisso=""): # il prefisso è usato per l'identificazione del nome.  
    for utente in clients:  
        utente.send(bytes(prefisso, "utf8")+msg)
```

Questa funzione viene chiamata quando si deve inviare messaggi a tutti gli utenti della Chat.

La funzione «cicla» all'interno del dizionario che contiene «i clients» e invia sul relativo socket il contenuto del messaggio inviato dal client mittente accompagnato dal **prefisso**.

Il **prefisso** è fornito in ingresso al momento della chiamata della funzione e corrisponde al nome dell'utente che ha inviato il messaggio.



Servizio CHAT – Lato Server

```
if __name__ == "__main__":
```

Questa semplice riga è usata per permettere al nostro codice di capire se verrà eseguito come script, o se è invece sarà invocato come modulo da un qualche programma per usare una o più delle sua funzioni e classi.

- Se eseguiamo questo script direttamente da Shell, ad esempio tramite PyCharm o da riga di comando, allora Python assegna alla variabile `__name__` la stringa `"__main__"`. In questo caso la condizione «*if*» risulta TRUE.
- Se invece stiamo invocando lo script come modulo dall'interno di un altro programma, alla variabile `__name__` verrà assegnato il nome dello script, quindi il nome del modulo.

NOTA: Utilizziamo `if __name__ == "__main__"` in tutti quei casi in cui vogliamo poter usare il nostro codice, sia come programma che come modulo.



Servizio CHAT – Lato Server

```
SERVER.listen(5)
print("In attesa di connessioni...")
ACCEPT_THREAD = Thread(target=accetta_connessioni_in_entrata)
ACCEPT_THREAD.start()
ACCEPT_THREAD.join()
SERVER.close()
```

Il modo più semplice di usare un thread è quello di creare un'istanza con una funzione target e invocare **start ()** per iniziarlo.

Per attendere fino a quando un thread ha completato il suo lavoro, si utilizza il metodo **join ()**. Usiamo il join() con ACCEPT_THREAD in modo che lo script principale attenda il completamento e non salti alla riga successiva, che chiude il server.

Potete trovare tutti i dettagli sui Thread() al seguente [link](#)



Lato CLIENT



Servizio CHAT – Lato Client

```
#----Connessione al Server----
HOST = input('Inserire il Server host: ')
PORT = input('Inserire la porta del server host: ')
if not PORT:
    PORT = 53000
else:
    PORT = int(PORT)

BUFSIZ = 1024
ADDR = (HOST, PORT)

client_socket = socket(AF_INET, SOCK_STREAM)
client_socket.connect(ADDR)

receive_thread = Thread(target=receive)
receive_thread.start()
# Avvia l'esecuzione della Finestra Chat.
tk.mainloop()
```

Una volta ottenuto l'indirizzo e la porta da parte dell'utente e creato un socket per connettersi al server, diamo inizio al thread per la ricezione dei messaggi e quindi al ciclo principale (la GUI).



Servizio CHAT – Lato Client

```
#!/usr/bin/env python3
"""Script relativa alla chat del client utilizzato per lanciare la GUI Tkinter."""
from socket import AF_INET, socket, SOCK_STREAM
from threading import Thread
import tkinter as tkt
```

Per scrivere la GUI, utilizziamo **Tkinter**, uno strumento incluso in Python.

Quindi oltre ad eseguire l'import dei principali moduli (socket e threading), eseguiamo l'import anche di **tkinter**.

Per ulteriori dettagli su Tkinter potete fare riferimento al seguente [link](#)



Servizio CHAT – Lato Client

```
"""La funzione che segue ha il compito di gestire la ricezione dei messaggi."""  
def receive():  
    while True:  
        try:  
            #quando viene chiamata la funzione receive, si mette in ascolto dei messaggi che  
            #arrivano sul socket  
            msg = client_socket.recv(BUFSIZ).decode("utf8")  
            #visualizziamo l'elenco dei messaggi sullo schermo  
            #e facciamo in modo che il cursore sia visibile al termine degli stessi  
            msg_list.insert(tkt.END, msg)  
            # Nel caso di errore e' probabile che il client abbia abbandonato la chat.  
        except OSError:  
            break
```

Perché un ciclo infinito?

Vogliamo ricevere messaggi quando siamo connessi e inviarli quando vogliamo. La funzionalità all'interno del loop è piuttosto semplice;

recv () è la parte bloccante. Termina l'esecuzione quando riceve un messaggio, e quindi prosegue aggiungendo il messaggio a **msg_list**.

msg_list è fondamentalmente una funzionalità di Tkinter per visualizzare l'elenco dei messaggi sullo schermo.

END (o "end") corrisponde alla posizione del cursore subito dopo l'ultimo carattere nel buffer.



Servizio CHAT – Lato Client

```
"""La funzione che segue gestisce l'invio dei messaggi."""
def send(event=None):
    # gli eventi vengono passati dai binders.
    msg = my_msg.get()
    # libera la casella di input.
    my_msg.set("")
    # invia il messaggio sul socket
    client_socket.send(bytes(msg, "utf8"))
    if msg == "{quit}":
        client_socket.close()
        finestra.quit()
```

Stiamo usando ***event*** come argomento perché viene implicitamente passato da Tkinter quando viene **premuto il pulsante di invio sulla GUI**.

my_msg è il campo di input sulla GUI e quindi estraiamo il messaggio da inviare associandolo alla variabile: ***msg = my_msg.get ()***.

Successivamente, cancelliamo il campo di input con ***my_msg.set*** e quindi inviamo il messaggio al server, che, come abbiamo visto in precedenza, trasmette questo messaggio a tutti i client (se non è un messaggio «quit»). Se si tratta di un messaggio «quit», chiudiamo il socket e quindi l'app GUI (tramite `top.quit ()`)



Servizio CHAT – Lato Client

```
"""La funzione che segue viene invocata quando viene chiusa la finestra della chat."""  
def on_closing(event=None):  
    my_msg.set("{quit}")  
    send()
```

Definiamo un'altra funzione, che verrà chiamata quando scegliamo di chiudere la finestra della GUI.

È una sorta di funzione di pulizia prima della chiusura e deve chiudere la connessione socket prima della chiusura della GUI.

NOTA:

Se si vuole semplicemente uscire dal programma, è possibile usare ***destroy()***.

Se invece si vuole eseguire un ciclo infinito senza distruggere la finestra Tkinter allora si usa ***quit()***



Servizio CHAT – Lato Client

```
finestra = tk.Tk()  
finestra.title("Chat_Laboratorio")
```

Ora iniziamo a creare la GUI, nel Main Space (cioè all'esterno di qualsiasi funzione).
Iniziamo definendo il widget di livello superiore e impostando il suo titolo:
«**Chat_Laboratorio**»



Servizio CHAT – Lato Client

```
#creiamo il Frame per contenere i messaggi
messages_frame = tkt.Frame(finestra)
#creiamo una variabile di tipo stringa per i messaggi da inviare.
my_msg = tkt.StringVar()
#indichiamo all'utente dove deve scrivere i suoi messaggi
my_msg.set("Scrivi qui i tuoi messaggi.")
#creiamo una scrollbar per navigare tra i messaggi precedenti.
scrollbar = tkt.Scrollbar(messages_frame)
```

Quindi creiamo un frame per contenere l'elenco dei messaggi.

Successivamente, creiamo una variabile stringa, principalmente per memorizzare il valore che otteniamo dal campo di input.

Abbiamo impostato il valore iniziale di quella variabile a «**Scrivi qui i tuoi messaggi**» per richiedere all'utente di scrivere il proprio messaggio in quello spazio.

Successivamente, creiamo una barra di scorrimento per scorrere questo frame di messaggi.



Servizio CHAT – Lato Client

```
# La parte seguente contiene i messaggi.  
msg_list = tkt.Listbox(messages_frame, height=15, width=50, yscrollcommand=scrollbar.set)  
scrollbar.pack(side=tkt.RIGHT, fill=tkt.Y)  
msg_list.pack(side=tkt.LEFT, fill=tkt.BOTH)  
msg_list.pack()  
messages_frame.pack()
```

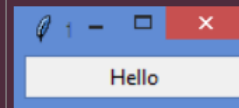
Ora definiamo l'elenco dei messaggi che verrà archiviato in ***message_frame*** e quindi impacchettato.

Potete trovare indicazioni sulla costruzione e gestione del Frame al seguente [link](#).

Examples

The message is displayed through Message() .

```
>>> import tkinter  
>>> from tkinter import*  
>>> master=tkinter.Tk()  
>>> message=Message(master, text="Hello")  
>>> message.pack()  
>>>
```



Servizio CHAT – Lato Client

```
#Creiamo il campo di input e lo associamo alla variabile stringa
entry_field = tkt.Entry(finestra, textvariable=my_msg)
# leggiamo la funzione send al tasto Return
entry_field.bind("<Return>", send)

entry_field.pack()
#creiamo il tasto invio e lo associamo alla funzione send
send_button = tkt.Button(finestra, text="Invio", command=send)
#integriamo il tasto nel pacchetto
send_button.pack()

finestra.protocol("WM_DELETE_WINDOW", on_closing)
```

- Creiamo il campo di input affinché l'utente possa inserire il proprio messaggio e lo associamo alla variabile stringa definita prima.
- Associamo il tasto RETURN alla funzione **send** () in modo che ogni volta che l'utente preme return, il messaggio viene inviato al server.
- Creiamo il pulsante di «**Invio**» nel caso in cui l'utente desideri inviare i propri messaggi facendo clic su di esso. Associamo questo pulsante alla funzione **send** ().
- Infine utilizziamo la funzione di pulizia **on_closing** () che dovrebbe essere chiamata quando l'utente desidera chiudere la finestra della GUI.



Servizio CHAT – Lato Client

Se infine intendete creare un file .exe eseguibile potete utilizzare, dopo averlo installato il pacchetto

PYINSTALLER

(da shell di DOS: *python -m pip install pyinstaller*)

In questo modo se fornite a Pyinstaller in ingresso il file relativo al client CHAT, potete creare un eseguibile da condividere con altre persone.



Esercizi



Esercizi – Livello di Applicazione e Livello di Trasporto



Esercizi - Livello di Applicazione



Esercizi - Livello di Applicazione

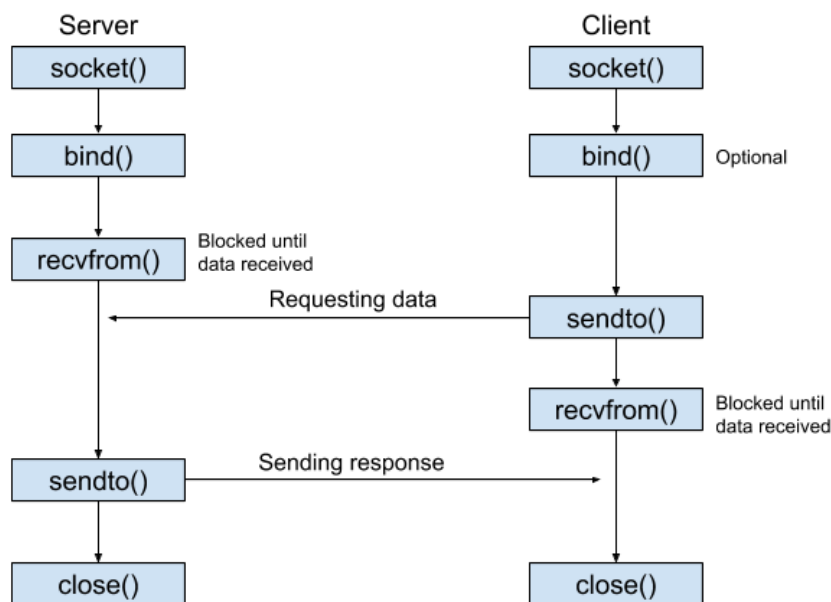
R26: Il server UDP, visto a lezione, richiedeva una sola socket, mentre il server TCP ne richiedeva due. Perché? Se il server TCP dovesse supportare n connessioni contemporanee, ciascuna proveniente da un diverso client, di quante socket avrebbe bisogno?



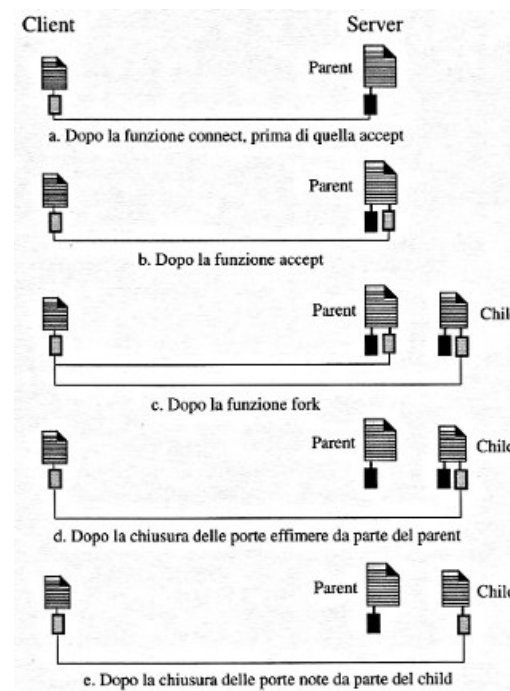
Soluzione Esercizio – R26

Con il server UDP, non esiste un socket di benvenuto e tutti i dati provenienti da client diversi entrano nel server attraverso questo socket.

Con il server TCP, esiste un socket di benvenuto e ogni volta che un client avvia una connessione al server, viene creato un nuovo socket. Quindi, per supportare n connessioni simultanee, il server avrebbe bisogno di $n+1$ socket.



Socket UDP



Socket TCP



Esercizi - Livello di Applicazione

P9: Si consideri la figura riportata qui di seguito, in cui si vede la rete di un'istituzione connessa a Internet. Supponete che la dimensione media degli oggetti sia di 850,000 bit e che la frequenza media di richieste dai browser dell'istituzione verso i server di origine sia di 16 richieste al secondo. Ipotizzate, inoltre, che la quantità di tempo che intercorre da quando il router sul lato Internet del collegamento di accesso inoltra una richiesta HTTP a quando riceve la risposta sia mediamente di **tre** secondi.

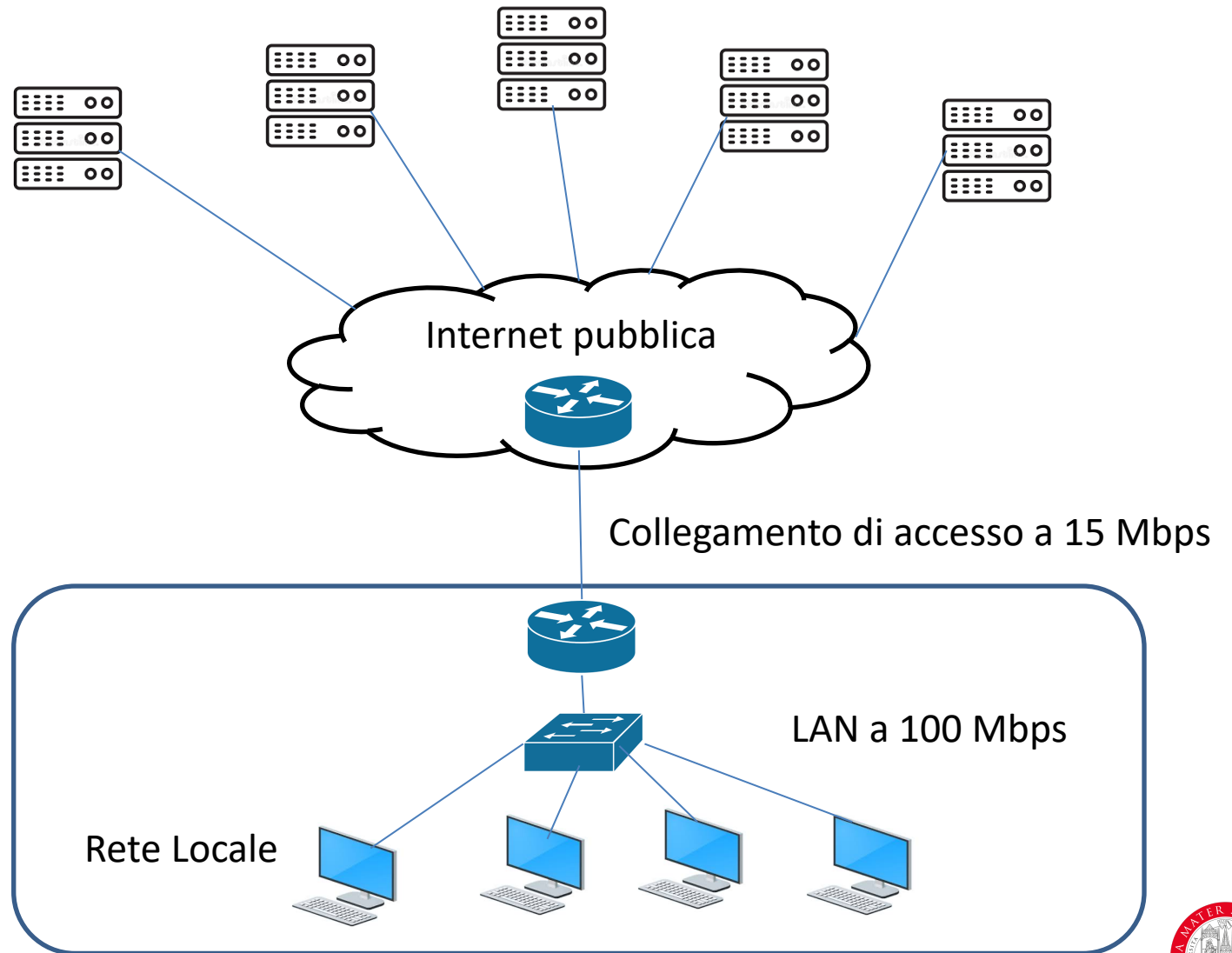
Fate un modello del tempo medio totale di risposta come la somma del ritardo medio di accesso (ossia, il ritardo dal router Internet al router dell'istituzione) e del ritardo Internet medio. Per il ritardo medio di accesso, usate la formula $\frac{\Delta}{(1-\Delta\beta)}$, dove Δ è il tempo medio richiesto per inviare un oggetto sul collegamento di accesso e β è la frequenza di arrivo di oggetti al collegamento di accesso.

- Trovate il tempo medio totale di risposta
- Supponete ora che nella LAN dell'istituzione sia installato un proxy e che la frequenza con cui gli oggetti non sono in cache (*miss rate*) sia 0.4. Trovate il tempo totale di risposta.



Esercizi - Livello di Applicazione

- dimensione media degli oggetti = 850,000 bit
- frequenza media di richieste dai browser dell'istituzione verso i server di origine sia di 16 richieste al secondo
- quantità di tempo che intercorre da quando il router sul lato Internet del collegamento di accesso inoltra una richiesta HTTP a quando riceve la risposta sia mediamente di tre secondi.
- Collegamento di accesso a 15 Mbps



Soluzione Esercizio P9.a

- a. Il tempo per trasmettere un oggetto di dimensione L su un collegamento di velocità R è $\frac{L}{R}$. Il tempo medio è la dimensione media dell'oggetto divisa per R ($= 15Mbps$ si veda il disegno):

$$\Delta = \frac{850,000 \text{ bits}}{15,000,000 \text{ bits/sec}} = 0.0567 \text{ sec}$$

L'intensità del traffico sul collegamento è data da

$$\beta\Delta = (16 \text{ richieste/sec}) \cdot (0.0567 \text{ sec/richiesta}) = 0.907$$

Pertanto, il ritardo medio di accesso è $\frac{\Delta}{(1-\Delta\beta)} = \frac{0.0567 \text{ sec}}{(1-0.907)} = 0.6 \text{ sec}$

Il tempo di risposta medio totale è quindi

$$T_{\text{medio totale di risposta}} = 0.6 \text{ sec} + 3 \text{ sec} = 3.6 \text{ sec}$$



Soluzione Esercizio P9.b

- b. L'intensità di traffico sul collegamento di accesso è ridotta del 60% in quanto il 60% delle richieste viene soddisfatta all'interno della rete istituzionale.

Pertanto il ritardo di accesso medio è

$$\frac{\Delta}{(1 - 0.4 \cdot \Delta\beta)} = \frac{0.0567 \text{ sec}}{(1 - 0.4 \cdot 0.907)} = 0.089 \text{ sec}$$

Il tempo di risposta è approssimativamente zero se la richiesta è soddisfatta dalla cache (cosa che avviene con probabilità 0.6);

il tempo di risposta medio è di $0.089 + 3 = 3.089 \text{ sec}$ per cache miss (che si verifica il 40% delle volte).

Quindi il tempo medio di risposta è $(0.6) \cdot (0 \text{ sec}) + (0.4) \cdot (3.089 \text{ sec}) = 1.24 \text{ sec}$.

Così il tempo medio di risposta è ridotto da 3,6 sec a 1,24 sec.



Esercizi - Livello di Applicazione

P10: Considerate un breve collegamento di 10 metri, sul quale il mittente può trasmettere a una frequenza di 150 bps in entrambe le direzioni. Supponete che i pacchetti dati siano lunghi 100,000 bit e quelli contenenti solo controllo (per esempio ACK o handshake) siano lunghi 200 bit.

Assumete che vi siano N connessioni parallele e ciascuna occupi $\frac{1}{N}$ della banda del collegamento.

Considerate ora il protocollo HTTP e supponete che ciascun oggetto scaricato sia lungo 100 kbit e che l'oggetto iniziale scaricato contenga 10 oggetti referenziati dallo stesso mittente.

Ha senso lo scaricamento parallelo tramite istanze parallele non persistenti di HTTP?

Considerate ora HTTP persistente. Vi aspettate un guadagno significativo rispetto al caso non persistente? Giustificate e spiegate la vostra risposta.



Soluzione Esercizio P10

Si noti che ogni oggetto scaricato (100Kbit) può essere completamente inserito in un pacchetto di dati (100,000 bit).

Sia T_p il ritardo di propagazione unidirezionale tra il client e il server.

Considera innanzitutto **i download paralleli che utilizzano connessioni non persistenti**. I download paralleli consentirebbero a 10 connessioni di condividere la larghezza di banda di 150 bit/sec, dando a ciascuna solo 15 bit/sec. Pertanto, il tempo totale necessario per ricevere tutti gli oggetti è dato da:

$$\left(\underbrace{\frac{200}{150} + T_p}_{\text{syn}} + \underbrace{\frac{200}{150} + T_p}_{\text{ack}} + \underbrace{\frac{200}{150} + T_p}_{\text{Syn ack}} + \underbrace{\frac{100,000}{150} + T_p}_{\text{Pagina html}} \right) + \left(\underbrace{\frac{200}{150/10} + T_p}_{\substack{10 \text{ syn} \\ \text{In parallelo}}} + \underbrace{\frac{200}{150/10} + T_p}_{\substack{10 \text{ ack} \\ \text{In parallelo}}} + \underbrace{\frac{200}{150/10} + T_p}_{\substack{10 \text{ syn ack} \\ \text{In parallelo}}} + \underbrace{\frac{100,000}{150/10} + T_p}_{\substack{10 \text{ oggetti} \\ \text{In parallelo}}} \right) = 7377 + 8 \cdot T_p \text{ sec}$$

Connessioni TCP NON Persistenti

$$1.33 + 1.33 + 1.33 + 666.67 + 13.3 + 13.3 + 13.3 + 6666.67 + 8 \cdot T_p = 7377.27 + 8 \cdot T_p$$



Soluzione Esercizio P10

Consideriamo ora **una connessione HTTP persistente**. Il tempo totale necessario è dato da:

$$\left(\underbrace{\frac{200}{150} + T_p}_{\text{syn}} + \underbrace{\frac{200}{150} + T_p}_{\text{ack}} + \underbrace{\frac{200}{150} + T_p}_{\text{Syn ack}} + \underbrace{\frac{100,000}{150} + T_p}_{\text{Pagina html}} \right) + 10 \cdot \left(\underbrace{\frac{200}{150} + T_p}_{\text{Richiesta oggetto}} + \underbrace{\frac{100,000}{150} + T_p}_{\text{Invio oggetto}} \right) = 7351 + 24 \cdot T_p \text{ sec}$$

Connessione TCP Persistente

$$1.33 + 1.33 + 1.33 + 666.67 + 13.3 + 6666.67 + 24 \cdot T_p = 7350.67 + 24 \cdot T_p$$

Supponendo che la velocità della luce sia $300 \cdot 10^6 m/s$, allora $T_p = \frac{10 m}{300 \cdot 10^6 m/s} = 0.03 \cdot 10^{-6} sec = 0.03 \mu s$.

T_p è quindi trascurabile rispetto al ritardo di trasmissione.

Pertanto, vediamo che l'HTTP persistente non è significativamente più veloce (meno dell'1%) rispetto al caso non persistente con download parallelo.



Esercizi - Livello di Applicazione

P22: Considerate la distribuzione di un file di dimensione $F = 15 \text{ Gbit}$ a N peer.

Il server ha una velocità di upload di $u_s = 30 \text{ Mbps}$ e ciascun peer ha una banda di download di $d_i = 2 \text{ Mbps}$ e di upload u .

Per $N = 10, 100, 1000$ e $u = 300 \text{ kbps}, 700 \text{ kbps}$ e 2 Mbps , preparate un grafico o una tabella che rappresenti per entrambi i metodi di distribuzione, client-server e P2P, il tempo di distribuzione minimo per ciascuna combinazione di N e u .

DEFINIZIONE: Il tempo di distribuzione è il tempo necessario perché tutti gli N peer ottengano una copia del file.



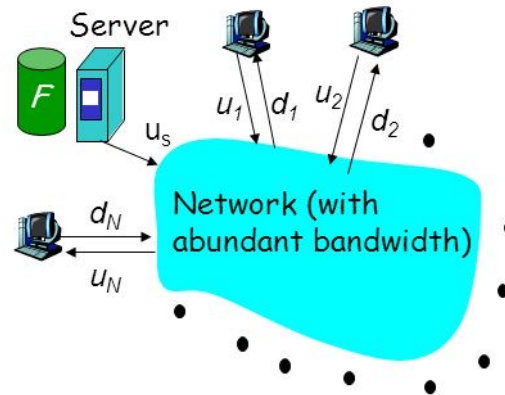
Soluzione Esercizio P22

Per calcolare il tempo minimo di distribuzione per la distribuzione client-server, utilizziamo la seguente formula:

$$D_{cs} = \max \left\{ NF/u_s, F/d_{\min} \right\}$$

File distribution time: server-client

- server sequentially sends N copies:
 - ❖ NF/u_s time
- client i takes F/d_i time to download



Time to distribute F to N clients using client/server approach $= d_{cs} = \max \{ NF/u_s, F/\min_i(d_i) \}$

increases linearly w.r.t. N (for large N)

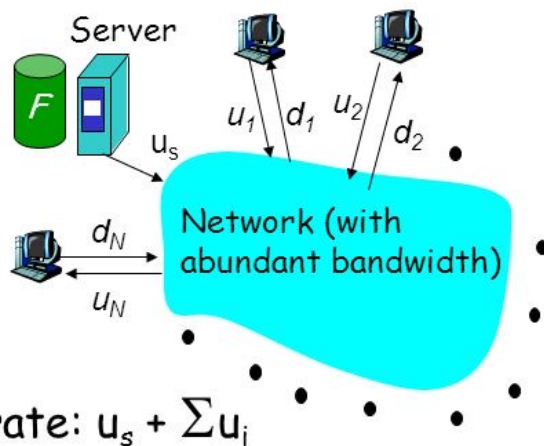
Soluzione Esercizio P22

Allo stesso modo, per calcolare il tempo minimo di distribuzione per la distribuzione P2P, utilizziamo la seguente formula:

$$D_{P2P} = \max \left\{ F/u_s, F/d_{\min}, NF / \left(u_s + \sum_{i=1}^N u_i \right) \right\}$$

File distribution time: P2P

- ❑ server must send one copy: F/u_s time
- ❑ client i takes F/d_i time to download
- ❑ NF bits must be downloaded (aggregate)
 - ❑ fastest possible upload rate: $u_s + \sum u_i$



Nell'architettura P2P, notiamo che:

- all'inizio della distribuzione, solo il server contiene il file. Per inviarlo alla comunità dei peer, deve effettuare l'upload almeno una volta attraverso il proprio link di accesso, quindi il minimo tempo di distribuzione lato server è almeno F/u_s . A differenza dell'architettura client-server, infatti, può non essere necessario più di un invio da parte del server, dato che i peer possono ridistribuire il file tra loro autonomamente
- come nell'architettura client-server, il peer con la connessione più lenta non può ricevere in un tempo minore di $F/d_{\min}$
- la capacità totale di upload del sistema nel suo insieme è pari alla velocità di upload del server, più le velocità di upload di ogni singolo peer, ossia $u_{\text{tot}} = u_s + u_1 + u_2 + \dots + u_n$. Poiché il sistema deve trasmettere globalmente NF bit, il tempo minimo di distribuzione non può essere inferiore a $NF/(u_s + u_1 + u_2 + \dots + u_n)$

$$d_{P2P} = \max \left\{ F/u_s, F/\min(d_i), NF/(u_s + \sum u_i) \right\}$$

Soluzione Esercizio P22

Dove:

$$F = 15Gbits = 15 * 1024 Mbits$$

$$u_s = 30Mbps$$

$$d_{min} = d_i = 2Mbps$$

Si noti inoltre che

$$300 Kbps = \frac{300}{1024} Mbps$$

Client - Server

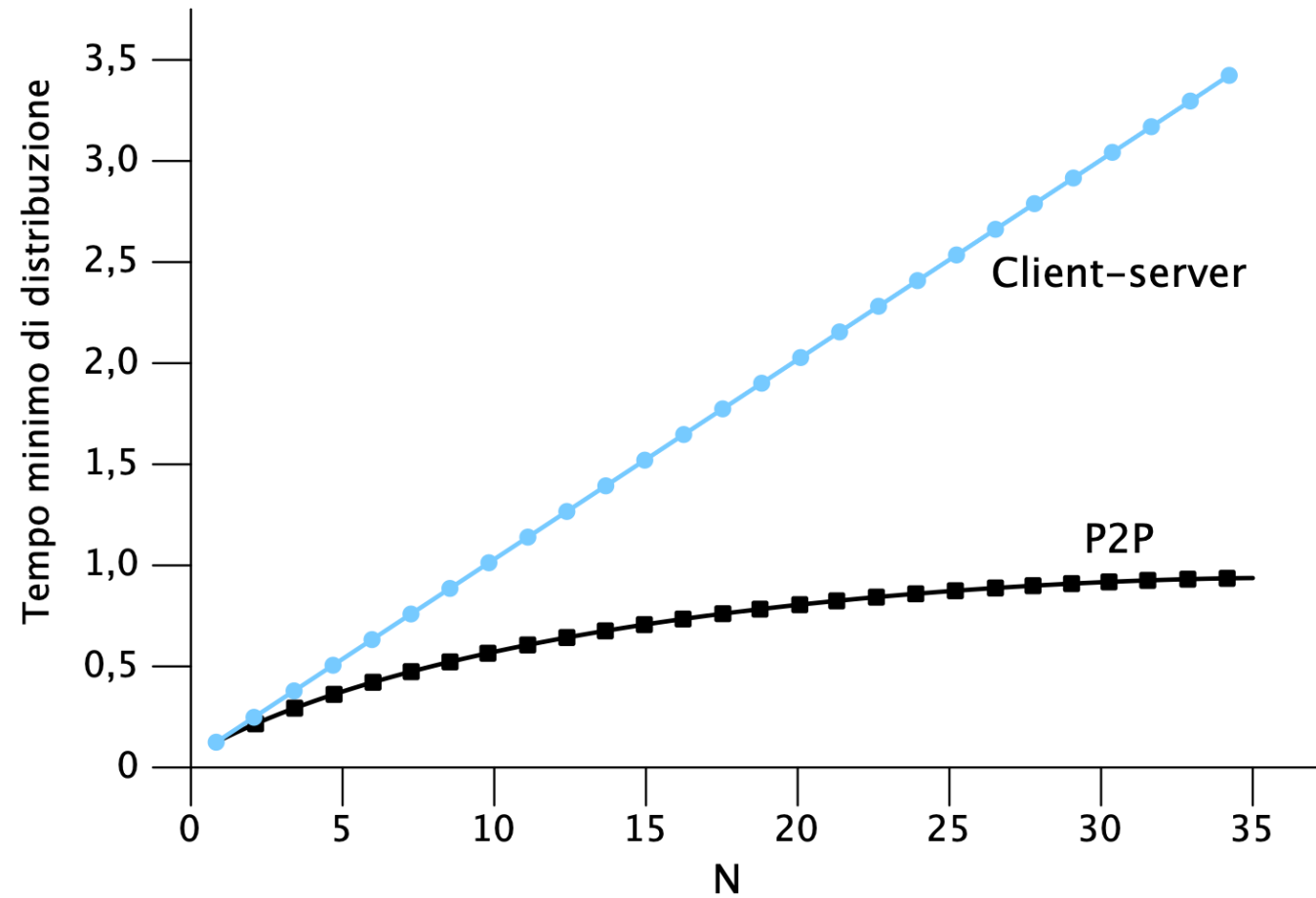
		N		
		10	100	1000
u	300 Kbps	7680	51200	512000
	700Kbps	7680	51200	512000
	2 Mbps	7680	51200	512000

Peer - Peer

		N		
		10	100	1000
u	300 Kbps	7680	25904	47559
	700Kbps	7680	15616	21525
	2 Mbps	7680	7680	7680



Soluzione Esercizio P22



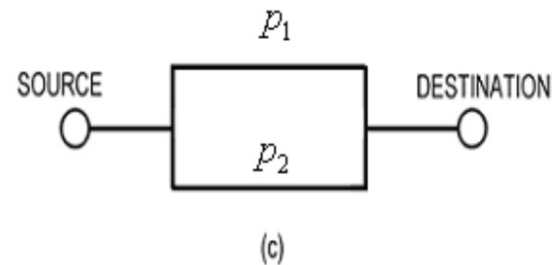
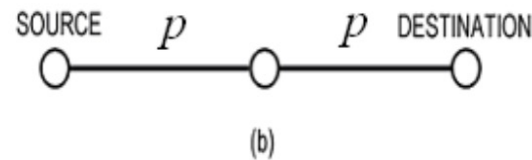
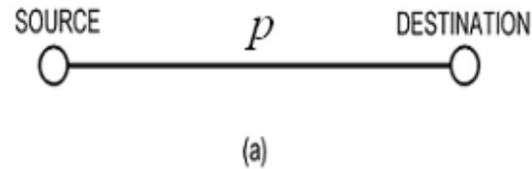
Esercizi



Esercizi

Consideriamo i modelli di rete mostrati in Figura.

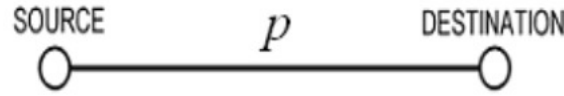
La probabilità che un pacchetto sia danneggiato su un collegamento di rete è p .



Basic network structures: (a) single link; (b) series link; (c) parallel link



Single-Link Network



Si consideri un messaggio di posta elettronica trasmesso su un unico collegamento di rete.

L'e-mail è suddivisa in K pacchetti, ognuno dei quali viene trasmesso individualmente attraverso il collegamento.

La probabilità che una trasmissione fallisca è p ; in caso di trasmissione fallita, il pacchetto viene ritrasmesso.

Introduciamo l'idea della distribuzione di Bernoulli di una variabile casuale osservando una singola trasmissione. Notiamo che il risultato di una singola trasmissione (la variabile casuale X) è un successo o un fallimento.



Quesito 1

Quale è la probabilità che il pacchetto sia trasmesso correttamente dopo n trasmissioni?

- ☐ $p^n \cdot (1 - p)$
- ☐ $p^{n-1} \cdot (1 - p)$
- ☐ p^n
- ☐ p^{n-1}



Quesito 1

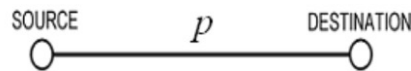
Quale è la probabilità che il pacchetto sia trasmesso correttamente dopo n trasmissioni?

- ☐ $p^n \cdot (1 - p)$
- ☒ $p^{n-1} \cdot (1 - p)$
- ☐ p^n
- ☐ p^{n-1}

Motivo: la trasmissione deve fallire $(n - 1)$ volte, ciascuna con probabilità p , e poi riuscire all'ennesimo tentativo.



Single-Link Network

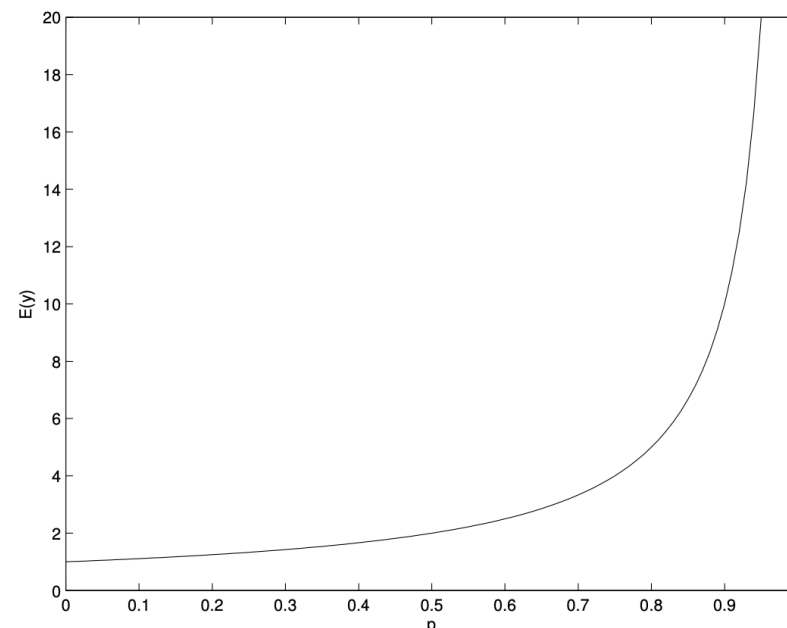


Definiamo la variabile casuale Y come il numero di trasmissioni per avere una trasmissione riuscita di un pacchetto.

Il numero medio di trasmissioni $E(Y)$ per una trasmissione riuscita di un pacchetto è:

$$E(Y) = \sum_{i=0}^{\infty} i p^{i-1} (1 - p) = \frac{1}{(1 - p)}$$

Un rapido sguardo alle condizioni al contorno mostra che se $p = 0$ (tutte le trasmissioni hanno esito positivo), allora il numero previsto di trasmissioni è 1, mentre se $p = 1$ (tutte le trasmissioni falliscono), sono necessarie infinite trasmissioni.



Quesito 2

Supponiamo di volere che il pacchetto attraversi il collegamento dopo non più di due tentativi. Quale è la probabilità che ciò accada?

☐ $q_{1,2} = (1 - 2p)$

☐ $q_{1,2} = (2p)$

☐ $q_{1,2} = (1 - p^2)$

☐ $q_{1,2} = p^2$



Quesito 2

Supponiamo di volere che il pacchetto attraversi il collegamento dopo non più di due tentativi. Quale è la probabilità che ciò accada?

☐ $q_{1,2} = (1 - 2p)$

☐ $q_{1,2} = (2p)$

☒ $q_{1,2} = (1 - p^2)$

☐ $q_{1,2} = p^2$

$$(1 - p) + p(1 - p) = (1 - p)(1 + p) = 1 - p^2$$

MOTIVO: è la somma delle probabilità che il pacchetto arrivi dopo esattamente uno o esattamente due tentativi o 1 meno la probabilità di due fallimenti in due tentativi). Ad esempio, se $p = 0$ ossia tutti i tentativi hanno successo, è necessaria solo una trasmissione. Tuttavia, se $p = 1$ ossia tutti i tentativi falliscono, la probabilità è zero.



Quesito 3

In generale quale è la probabilità che un pacchetto venga trasmesso con successo attraverso una rete **single-link** al massimo in N tentativi?

- ☐ $q_{1,N} = 1 - p^N$
- ☐ $q_{1,N} = 1 - p^{N-1}$
- ☐ $q_{1,N} = p^N$
- ☐ $q_{1,N} = p^{N+1}$



Quesito 3

In generale quale è la probabilità che un pacchetto venga trasmesso con successo attraverso una rete **single-link** al massimo in N tentativi?



$$q_{1,N} = 1 - p^N$$

$$q_{1,N} = \sum_{i=1}^N p^{i-1}(1-p) = 1 - p^N$$



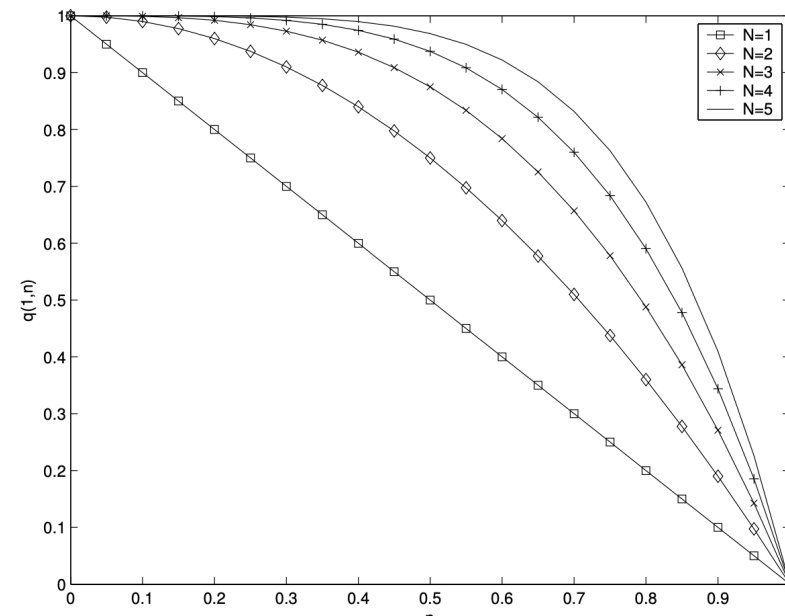
$$q_{1,N} = 1 - p^{N-1}$$



$$q_{1,N} = p^N$$



$$q_{1,N} = p^{N+1}$$



OSSERVAZIONE: per valori di p inferiori a uno (una probabilità diversa da zero di una singola trasmissione riuscita), la probabilità che il pacchetto venga eventualmente trasmesso con successo si avvicina a uno quando N aumenta (tende all'infinito).



Single Link: Multiple packets

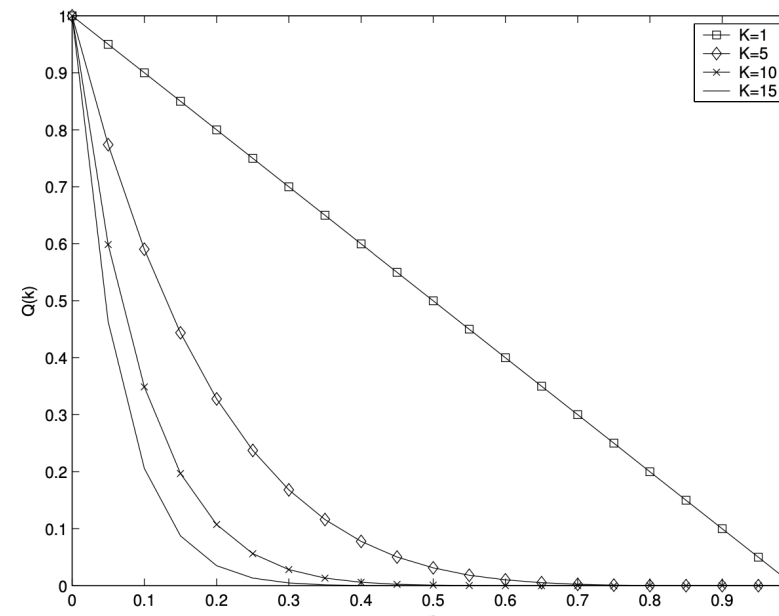
Un messaggio di posta elettronica viene suddiviso in più pacchetti prima di essere trasmesso in rete.
Supponiamo che ci siano K pacchetti, ognuno dei quali viene trasmesso separatamente.
A seconda dei valori di p e K , potrebbe essere molto improbabile che l'intero messaggio venga trasmesso correttamente al primo tentativo. Tutte le trasmissioni di pacchetti devono riuscire, rendendo questa probabilità pari a:

Un messaggio di posta elettronica viene suddiviso in più pacchetti prima di essere trasmesso in rete.

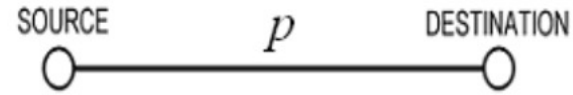
Supponiamo che ci siano K pacchetti, ognuno dei quali viene trasmesso separatamente.

A seconda dei valori di p e K , potrebbe essere molto improbabile che l'intero messaggio venga trasmesso correttamente al primo tentativo. Tutte le trasmissioni di pacchetti devono riuscire, rendendo questa probabilità pari a:

$$Q_K = (1 - p)^K$$



Single Link: Multiple packets



Supponiamo che i pacchetti vengano inviati sulla rete in successione e, se una trasmissione di pacchetti fallisce, il pacchetto fallito viene ritrasmesso fino a quando non si verifica il successo.

I pacchetti trasmessi correttamente prima che un pacchetto successivo fallisca non vengono reinviati.

Definiamo la variabile casuale Z come il numero medio di trasmissioni richieste per una corretta trasmissione dell'intera e-mail.



Quesito 4

Quale è il valore atteso di trasmissioni, $E(Z)$, rispetto a p e K ?

☐ $(1 - p)^K$

☐ $(1 - p)^{K-1}$

☐ $\frac{K}{(1 - p)}$

☐ $\frac{K}{(1 - p)^{K-1}}$

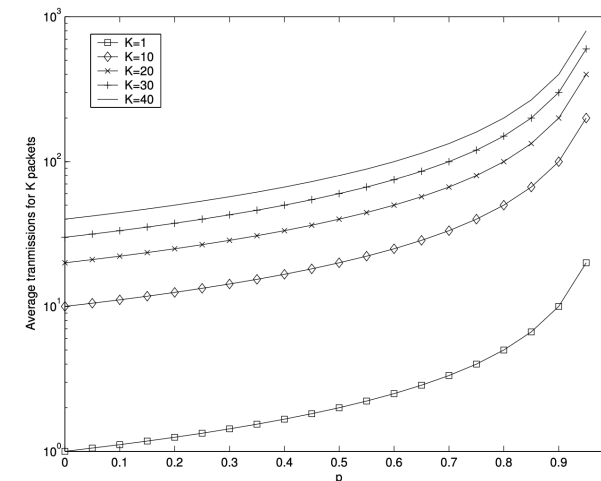


Quesito 4

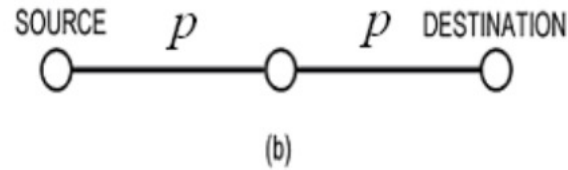
Quale è il valore atteso di trasmissioni, $E(Z)$, rispetto a p e K ?

- ☐ $(1 - p)^K$
- ☐ $(1 - p)^{K-1}$
- ☒ $\frac{K}{(1 - p)}$
- ☐ $\frac{K}{(1 - p)^{K-1}}$

MOTIVO: $E(Z)$ può essere calcolato facilmente poiché il successo o il fallimento di qualsiasi trasmissione di pacchetto è indipendente dalle altre $(K - 1)$ trasmissioni, e abbiamo visto nella slide 7 che un singolo pacchetto richiede (in media) $\frac{1}{(1-p)}$ trasmissioni per avere successo, quindi ci aspettiamo che l'insieme completo di K pacchetti venga trasmesso in $\frac{K}{(1-p)}$ tentativi.



Series Two-Link Network



Si supponga che ciascun pacchetto dell'e-mail suddivisa in K pacchetti debba essere trasmesso attraverso due collegamenti di rete collegati in serie.

La probabilità di perdere un pacchetto su qualsiasi collegamento è ancora p .

Durante il processo di trasmissione, se un pacchetto è danneggiato su uno qualsiasi dei due collegamenti, deve essere ritrasmesso nuovamente attraverso il primo collegamento.



Quesito 5

Quale è la probabilità che un pacchetto venga trasmesso correttamente su entrambi i collegamenti nella rete?

☐ $1 - p^2$

☐ $(1 - p)^2$

☐ $(1 - p)p$

☐ $(1 - p)$



Quesito 5

Quale è la probabilità che un pacchetto venga trasmesso correttamente su entrambi i collegamenti nella rete?

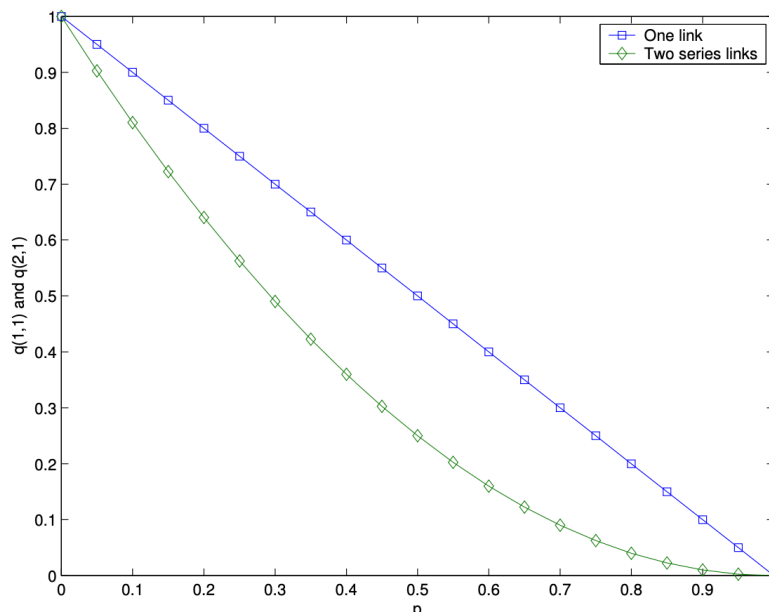
☐ $1 - p^2$

☒ $(1 - p)^2$

☐ $(1 - p)p$

☐ $(1 - p)$

MOTIVO: La probabilità che un pacchetto venga trasmesso con successo attraverso entrambi i collegamenti nella rete è $(1 - p)^2$ perché una trasmissione riuscita con probabilità $(1-p)$ deve avere successo in due prove consecutive indipendenti.



Quesito 6

Quale è la probabilità che una mail composta da K pacchetti venga trasmessa correttamente su questa rete al primo tentativo?

☐ $(1 - p)^{K+2}$

☐ $(1 - p)^{K^2}$

☐ $(1 - p)^{2K}$

☐ $K \cdot (1 - p)^2$



Quesito 6

Quale è la probabilità che una mail composta da K pacchetti venga trasmessa correttamente su questa rete al primo tentativo?

MOTIVO: tutti i K pacchetti devono essere trasmessi correttamente sui due collegamenti ossia un totale di $2K$ trasmissioni riuscite.

☐ $(1 - p)^{K+2}$

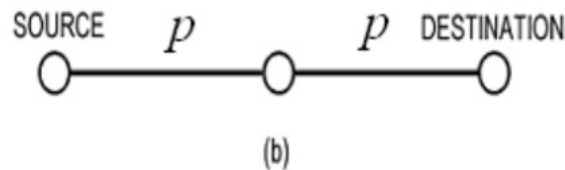
☐ $(1 - p)^{K^2}$

☒ $(1 - p)^{2K}$ ← $(1 - p)^K \cdot (1 - p)^K$

☐ $K \cdot (1 - p)^2$



Series Two-Link Network



Sia Z la variabile casuale che rappresenta il numero di volte in cui un pacchetto viene trasmesso sul primo collegamento fino a quando non viene ricevuto correttamente.

Se la trasmissione di un pacchetto fallisce (con probabilità p), dobbiamo inviare nuovamente il pacchetto attraverso entrambi i collegamenti.

Sia $q_{2,k}$ la probabilità che il pacchetto venga trasmesso con successo esattamente in k tentativi.

Allora $q_{2,k}$ può essere espresso con la seguente ricorsione:

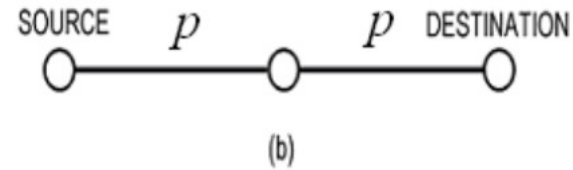
$$q_{2,1} = (1 - p)^2$$

$$q_{2,k} = (1 - \sum_{j=1}^{k-1} q_{2,j})(1 - p)^2$$

Notiamo che affinché la trasmissione abbia successo al k – *esimo* tentativo, tutti i tentativi precedenti devono essere falliti. La probabilità che il tentativo sia riuscito per qualche $j < k$ è $\sum_{j=1}^{k-1} q_{2,j}$ quindi la probabilità che esso fallisca $1 - \sum_{j=1}^{k-1} q_{2,j}$. Infine, il k – *esimo* tentativo deve riuscire, quindi abbiamo il termine $(1 - p)^2$ alla fine.



Series Two-Link Network



Attraverso qualche manipolazione possiamo ricavare una forma esplicita, non ricorsiva.

$$q_{2,k} = (1 - \sum_{j=1}^{k-1} q_{2,j})(1 - p)^2$$

$$q_{2,k} = (1 - p)^2 - (1 - p)^2 \sum_{j=1}^{k-1} q_{2,j}$$

$$q_{2,k} = (1 - p)^2 - (1 - p)^2 \sum_{j=1}^{k-2} q_{2,j} - (1 - p)^2 q_{2,k-1}$$

$$q_{2,k} = q_{2,k-1} - (1 - p)^2 q_{2,k-1}$$

$$q_{2,k} = q_{2,k-1}(1 - (1 - p)^2)$$

$$q_{2,k} = q_{2,k-1}(1 - (1 - p)^2)$$

$$q_{2,k} = q_{2,2}(1 - (1 - p)^2)^{k-1}$$

$$q_{2,k} = (1 - p)^2(1 - (1 - p)^2)^{k-1}$$



Quesito 7

Qual è il numero medio di trasmissioni sul primo collegamento, $E(Z)$, necessario per inviare un singolo pacchetto attraverso questa rete?

☐ $\frac{1}{(1 - p^2)}$

☐ $\frac{1}{(1 - p)^k}$

☐ $\frac{1}{(1 - p)^2}$

☐ $(1 - p)^k$



Quesito 7

Qual è il numero medio di trasmissioni sul primo collegamento, $E(Z)$, necessario per inviare un singolo pacchetto attraverso questa rete?

☐ $\frac{1}{(1-p^2)}$

☐ $\frac{1}{(1-p)^k}$

☒ $\frac{1}{(1-p)^2}$

☐ $(1-p)^k$

Vedi slide 7

$$E(Z) = \sum_{k=1}^{\infty} k (1-p)^2 (1-(1-p)^2)^{k-1}$$
$$E(Z) = \sum_{k=1}^{\infty} k \delta (1-\delta)^{k-1} = \delta \sum_{k=1}^{\infty} k (1-\delta)^{k-1} = \delta \frac{1}{\delta^2} = \frac{1}{\delta} = \frac{1}{(1-p)^2}$$



Series Two-Link Network

Tuttavia, la nostra e-mail ha K pacchetti separati, ognuno dei quali richiede una media di $\frac{1}{(1-p)^2}$ trasmissioni (supponiamo ancora che un pacchetto non venga trasmesso fino a quando il pacchetto precedente non ha attraversato la rete con successo).



Quesito 8

Quale è il numero totale atteso di trasmissioni di pacchetti previste per l'e-mail? .

☐ $\frac{K}{(1 - p^2)}$

☐ $\frac{1}{(1 - p)^K}$

☐ $\frac{K}{(1 - p)^2}$

☐ $(1 - p)^K$



Quesito 8

Quale è il numero totale atteso di trasmissioni di pacchetti previste per l'e-mail? .

☐ $\frac{K}{(1 - p^2)}$

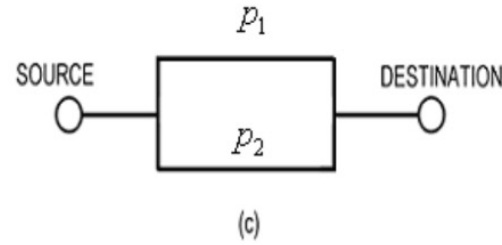
☐ $\frac{1}{(1 - p)^K}$

☒ $\frac{K}{(1 - p)^2}$

☐ $(1 - p)^K$



Parallel Two-Link Network



Supponiamo che la nostra rete sia costituita da due collegamenti che collegano una coppia di nodi comuni (una rete parallela).

La probabilità che un pacchetto venga danneggiato durante una trasmissione attraverso i due collegamenti è rispettivamente p_a e p_b .

Il pacchetto viaggia attraverso entrambi i collegamenti per arrivare a destinazione.



Quesito 9

Si consideri il caso in cui la trasmissione deve avere successo su entrambi i collegamenti contemporaneamente

Quale è la probabilità con cui il pacchetto raggiungerà la sua destinazione attraverso entrambi i collegamenti ?

- ☐ $(1 - p_a)p_b$
- ☐ $(1 - p_b)p_a$
- ☐ $(1 - p_a)(1 - p_b)$
- ☐ p_ap_b



Quesito 9

Quale è la probabilità con cui il pacchetto raggiungerà la sua destinazione attraverso entrambi i collegamenti (poiché la trasmissione deve avere successo su entrambi i collegamenti contemporaneamente)?

- ☐ $(1 - p_a)p_b$
- ☐ $(1 - p_b)p_a$
- ☒ $(1 - p_a)(1 - p_b)$
- ☐ p_ap_b



Quesito 10

Si consideri il caso in cui la trasmissione del pacchetto è considerata riuscita se il pacchetto viaggia su uno dei due collegamenti paralleli senza danneggiarsi.

Quale è la probabilità con cui il pacchetto raggiungerà la sua destinazione?

☐ $(1 - p_a p_b)$

☐ $(1 - p_b) p_a$

☐ $(1 - p_a)(1 - p_b)$

☐ $p_a p_b$



Quesito 10

Si consideri il caso in cui la trasmissione del pacchetto è considerata riuscita se il pacchetto viaggia su uno dei due collegamenti paralleli senza danneggiarsi.

Quale è la probabilità con cui il pacchetto raggiungerà la sua destinazione?

☒ $(1 - p_a p_b)$

☐ $(1 - p_b)p_a$

☐ $(1 - p_a)(1 - p_b)$

☐ $p_a p_b$

MOTIVO: Ciò si verifica con probabilità $(1 - p_a p_b)$ perché l'unica possibilità che la trasmissione non riesca è se si verifica un errore in entrambi i collegamenti (che si verifica con probabilità $p_a p_b$).



DNS AND HTTP DELAYS

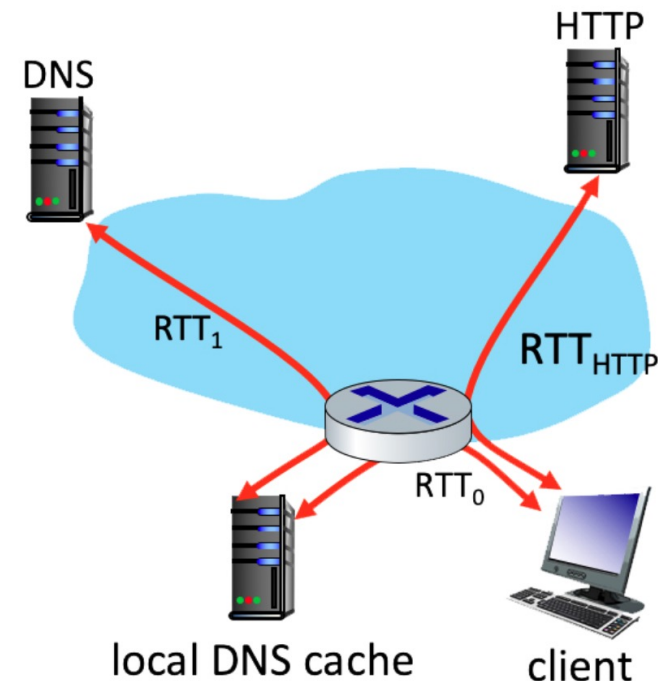
Si supponga di fare clic su un collegamento all'interno del browser Web per ottenere una pagina Web.

L'indirizzo IP per l'URL associato non è memorizzato nella cache dell'host locale, quindi è necessaria una ricerca DNS per ottenere l'indirizzo IP.

Supponiamo che due server DNS vengano visitati prima che il vostro host riceva l'indirizzo IP dal DNS.

Il primo server DNS visitato è la cache DNS locale, con un ritardo RTT di $RTT_0 = 2\text{ ms}$. Il secondo server DNS contattato ha un RTT di 2 ms .

Supponiamo inizialmente che la pagina Web associata al collegamento contenga esattamente un oggetto, costituito da una piccola quantità di testo HTML. Si supponga che l'RTT tra l'host locale e il server Web contenente l'oggetto sia $RTT_{HTTP} = 95\text{ ms}$.



Quesito 11

Assumendo un tempo di trasmissione zero per l'oggetto HTML, quanto tempo (in msec) trascorre da quando il client fa clic sul collegamento fino a quando il client riceve l'oggetto?

☐ 97 *ms*

☐ 26 *ms*

☐ 194 *ms*

☐ 99 *ms*



Quesito 11

Assumendo un tempo di trasmissione zero per l'oggetto HTML, quanto tempo (in msec) trascorre da quando il client fa clic sul collegamento fino a quando il client riceve l'oggetto?

☐ 97 ms

☐ 26 ms

☒ 194 ms

☐ 99 ms

Il tempo da quando la richiesta Web viene effettuata nel browser fino alla visualizzazione della pagina nel browser è:
 $RTT_0 + RTT_1 + 2 \cdot RTT_{HTTP} = 2 + 2 + 2 \cdot 95 = 194 \text{ ms}.$

Si noti che sono necessari 2 RTT_{HTTP} per recuperare l'oggetto HTML: un RTT_{HTTP} per stabilire la connessione TCP e quindi un RTT_{HTTP} per eseguire HTTP GET/risposta su quella connessione TCP.



Quesito 12

Supponiamo ora che l'oggetto HTML faccia riferimento a 5 oggetti molto piccoli sullo stesso server.

Trascurando i tempi di trasmissione, quanto tempo (in msec) trascorre da quando il client fa clic sul collegamento fino a quando l'oggetto di base e tutti i 5 oggetti aggiuntivi vengono ricevuti dal server Web sul client, supponendo HTTP non persistente e nessuna connessione TCP parallela?

- ☐ 95 *ms*
- ☐ 752 *ms*
- ☐ 1144 *ms*
- ☐ 212 *ms*



Quesito 12

Supponiamo ora che l'oggetto HTML faccia riferimento a 5 oggetti molto piccoli sullo stesso server.

Trascurando i tempi di trasmissione, quanto tempo (in msec) trascorre da quando il client fa clic sul collegamento fino a quando l'oggetto di base e tutti i 5 oggetti aggiuntivi vengono ricevuti dal server Web sul client, supponendo HTTP non persistente e nessuna connessione TCP parallela?

☐ 95 ms

☐ 752 ms

☒ 1144 ms

☐ 212 ms

Il tempo da quando la richiesta Web viene effettuata nel browser fino alla visualizzazione della pagina nel browser è:

$$RTT_0 + RTT_1 + 2 \cdot RTT_{HTTP} + 2 \cdot 5 \cdot RTT_{HTTP} = 2 + 2 + 2 \cdot 95 + 2 \cdot 5 \cdot 95 = 1144 \text{ ms.}$$

Si noti che sono necessari due ritardi RTT_{HTTP} per recuperare l'oggetto HTML di base: un RTT_{HTTP} per stabilire la connessione TCP e un RTT_{HTTP} per inviare la richiesta HTTP e ricevere la risposta HTTP. Quindi, in serie, per ciascuno dei 5 oggetti incorporati, è necessario un ritardo di $2 \cdot RTT_{HTTP}$: un RTT_{HTTP} per stabilire la connessione TCP e quindi un RTT_{HTTP} per eseguire HTTP GET/risposta su quella connessione TCP.



Quesito 13

Si supponga che l'oggetto HTML faccia riferimento a 5 oggetti molto piccoli sullo stesso server, ma si assuma che il client sia configurato per supportare un massimo di 5 connessioni TCP parallele, con HTTP non persistente.

☐ 380 *ms*

☐ 384 *ms*

☐ 190 *ms*

☐ 212 *ms*



Quesito 13

Si supponga che l'oggetto HTML faccia riferimento a 5 oggetti molto piccoli sullo stesso server, ma si assuma che il client sia configurato per supportare un massimo di 5 connessioni TCP parallele, con HTTP non persistente.

☐ 380 *ms*

Poiché ci sono solo 5 oggetti, c'è un ritardo di 4 msec per la query DNS, due RTT_{HTTP} per la pagina di base e $2 \cdot RTT_{HTTP}$ per gli oggetti poiché le richieste per questi possono essere eseguite in parallelo.

☒ 384 *ms*

$$RTT_0 + RTT_1 + 2 \cdot RTT_{HTTP} + 2 \cdot RTT_{HTTP}$$

Il totale è $2 + 2 + 190 + 190 = 384 \text{ ms}$. Come sopra, sono necessari 2 RTT_{HTTP} per recuperare l'oggetto HTML di base: un RTT_{HTTP} per stabilire la connessione TCP e un RTT_{HTTP} per inviare la richiesta HTTP e ricevere la risposta HTTP contenente l'oggetto HTML di base.

☐ 190 *ms*

Una volta ricevuto l'oggetto di base sul client, i 5 HTTP Gets per gli oggetti incorporati possono procedere in parallelo. Ciascuno (in parallelo) richiede due ritardi RTT_{HTTP} : uno RTT_{HTTP} per configurare la connessione TCP e un RTT_{HTTP} per eseguire HTTP GET/risposta per un oggetto incorporato.

☐ 212 *ms*



Quesito 14

Si supponga che l'oggetto HTML faccia riferimento a 5 oggetti molto piccoli sullo stesso server, ma si assuma che il client sia configurato per supportare un massimo di 5 connessioni TCP parallele, con HTTP persistente.

☐ 280 *ms*

☐ 289 *ms*

☐ 412 *ms*

☐ 212 *ms*



Quesito 14

Si supponga che l'oggetto HTML faccia riferimento a 5 oggetti molto piccoli sullo stesso server, ma si assuma che il client sia configurato per supportare un massimo di 5 connessioni TCP parallele, con HTTP persistente.

☐ 280 ms

☒ 289 ms

☐ 412 ms

☐ 212 ms

Poiché ci sono solo 5 oggetti, c'è un ritardo di 4 msec per la query DNS, due RTT_{HTTP} per la pagina di base e 1 RTT_{HTTP} per gli oggetti.

$$RTT_0 + RTT_1 + 2 \cdot RTT_{HTTP} + RTT_{HTTP}$$

Il totale è

$$2 + 2 + 190 + 95 = 289 \text{ ms}$$

Come sopra, sono necessari due ritardi RTT_{HTTP} per recuperare l'oggetto HTML di base: un RTT_{HTTP} per stabilire la connessione TCP e un RTT_{HTTP} per inviare la richiesta HTTP e ricevere la risposta HTTP contenente l'oggetto HTML di base. Tuttavia, con HTTP persistente, questa connessione TCP rimarrà aperta per future richieste HTTP, che quindi non subiranno un ritardo nella creazione del TCP. Una volta ricevuto l'oggetto base presso il client, è possibile procedere in parallelo al massimo di cinque richieste, ognuna delle quali recupera uno dei 5 oggetti incorporati. Ciascuno (in parallelo) richiede solo un ritardo RTT_{HTTP} per eseguire HTTP GET/risposta per un oggetto incorporato. Una volta recuperati questi primi cinque oggetti, (se necessario) è possibile recuperare (in parallelo) gli oggetti incorporati rimanenti. Questo secondo round di HTTP GET/risposta per recuperare gli oggetti incorporati rimanenti richiede solo un altro RTT_{HTTP} , poiché la connessione TCP è rimasta aperta.



Quesito 15

Qual è il metodo più veloce che abbiamo esplorato: seriale non persistente, parallelo non persistente o parallelo persistente?

☐ connessioni parallele non persistenti

☐ connessioni parallele persistenti

☐ connessioni seriali non persistenti



Quesito 15

Qual è il metodo più veloce che abbiamo esplorato: seriale non persistente, parallelo non persistente o parallelo persistente?

☐ connessioni parallele non persistenti

☒ connessioni parallele persistenti

☐ connessioni seriali non persistenti

Il ritardo quando si utilizzano connessioni parallele persistenti è inferiore rispetto all'utilizzo di connessioni parallele non persistenti, che è comunque più veloce rispetto all'utilizzo di connessioni seriali non persistenti.



Esercizi – Livello di Applicazione e Livello di Trasporto



Esercizi - Livello di Trasporto



Esercizi - Livello di Trasporto

R1: Supponete che il livello di rete fornisca il seguente servizio. Il livello di rete nell'host sorgente accetta un segmento di dimensione massima di 1200 byte e un indirizzo dell'host di destinazione dal livello di trasporto. Il livello di rete poi garantisce di consegnare il segmento al livello di trasporto nell'host di destinazione. Supponete che molti processi applicativi di rete possano essere in esecuzione sull'host di destinazione.

- a. Progettate un protocollo a livello di trasporto più semplice possibile che porti i dati applicativi al processo desiderato dell'host di destinazione. Assumete che il sistema operativo nell'host di destinazione abbia assegnato un numero di porta di 4 byte a ciascun processo applicativo in esecuzione.
- b. Modificate questo protocollo in modo che fornisca un «indirizzo di ritorno» al processo di destinazione.
- c. Nei vostri protocolli il livello di trasporto deve fare qualcosa nel nucleo della rete di calcolatori?



Esercizi - Livello di Trasporto

R8: Supponete che un web server sia in esecuzione sull'host C sulla porta 80.

Supponete che questo web server usi le connessioni persistenti e stia al momento ricevendo richieste da due diversi host, A e B.

Tutte le richieste vengono inviate attraverso la stessa socket sull'host C?

Se vengono fatte passare attraverso socket diverse, entrambe hanno la porta 80?

Discutete e spiegate questa situazione.



Soluzione Esercizio R8

Per ogni connessione persistente, il server Web crea un "socket di connessione" separato.

Ogni socket di connessione è identificato da una quadrupla: (indirizzo IP di origine, numero di porta di origine, indirizzo IP di destinazione, numero di porta di destinazione).

Quando l'host C riceve un datagramma IP, esamina questi quattro campi nel datagramma/segmento per determinare a quale socket deve passare il payload del segmento TCP.

Pertanto, le richieste di A e B passano attraverso socket diversi.

L'identificatore per entrambi questi socket ha 80 per la porta di destinazione; tuttavia, gli identificatori per questi socket hanno valori diversi per gli indirizzi IP di origine.

A differenza di UDP, quando il livello di trasporto passa il carico utile di un segmento TCP al processo dell'applicazione, non specifica l'indirizzo IP di origine, poiché questo è implicitamente specificato dall'identificatore del socket.



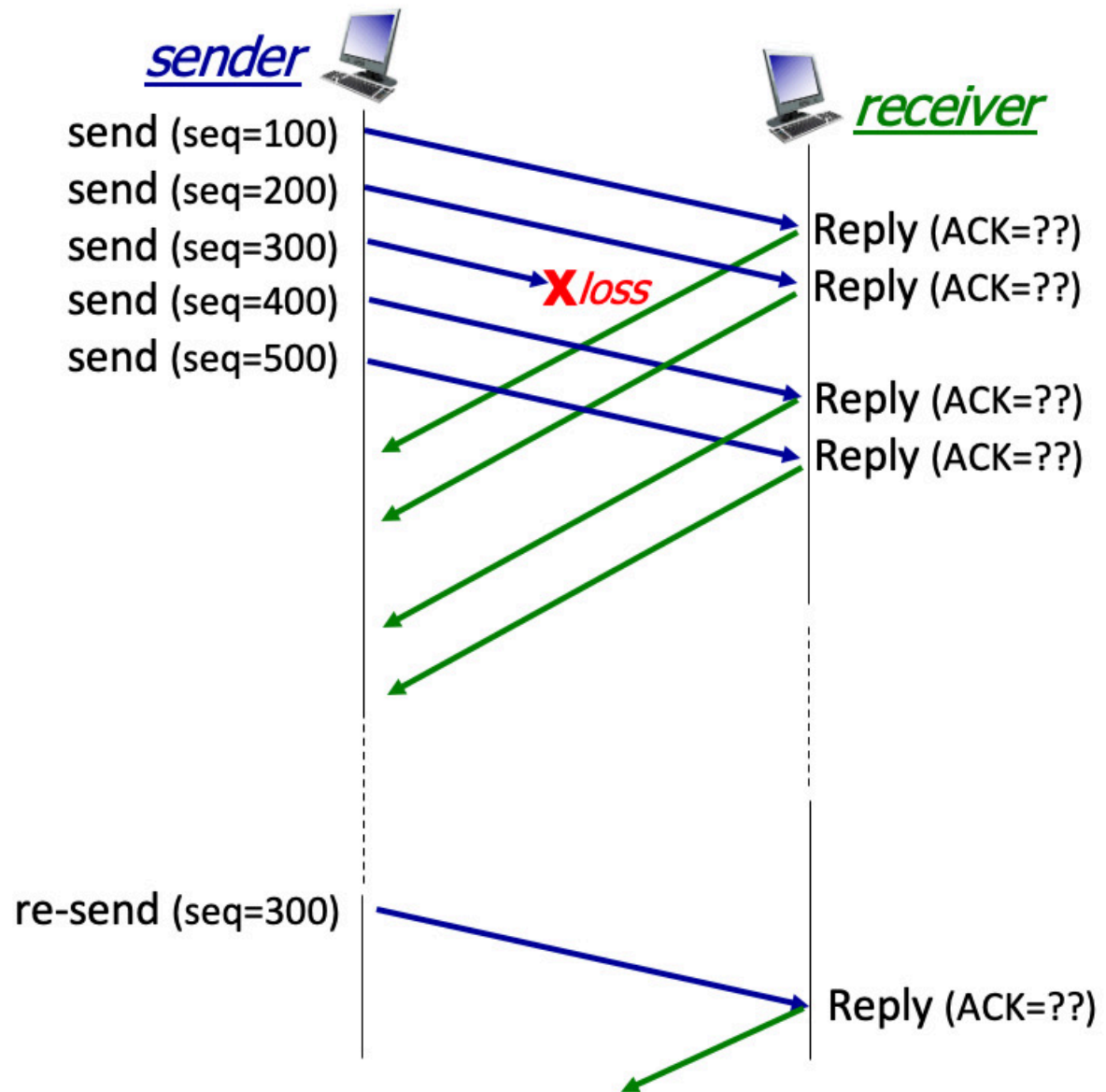
Esercizi - Livello di Trasporto

R9: Nei nostri protocolli *rdt*, perché abbiamo bisogno di introdurre i numeri di sequenza?



Soluzione Esercizio R9

I numeri di sequenza sono necessari a un destinatario per scoprire se un pacchetto in arrivo contiene nuovi dati o è una ritrasmissione.



Esercizi - Livello di Trasporto

P3: UDP e TCP utilizzano il complemento a 1 per calcolare il checksum. Supponiamo di avere i seguenti tre byte: 01010011, 01100110 e 01110100.

Qual è il complemento a 1 della loro somma?

Notiamo che, sebbene UDP e TCP usino checksum da 16 bit, in questo problema consideriamo addendi a 8 bit.

Con lo schema del complemento a 1, come vengono rilevati gli errori dal destinatario?

E' possibile che un errore su 1 bit non venga rilevato?

E che cosa avviene per gli errori su 2 bit?

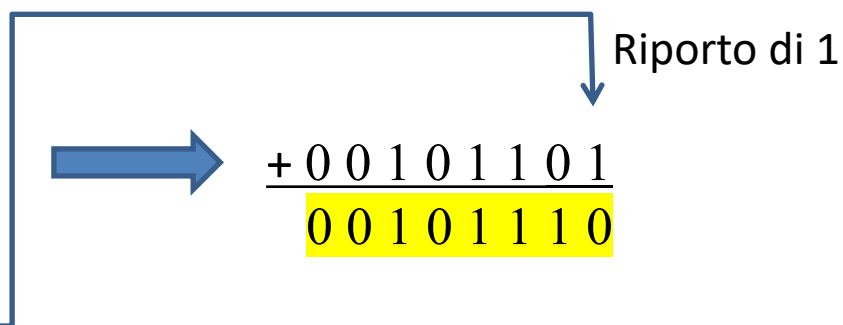


Soluzione Esercizio P3

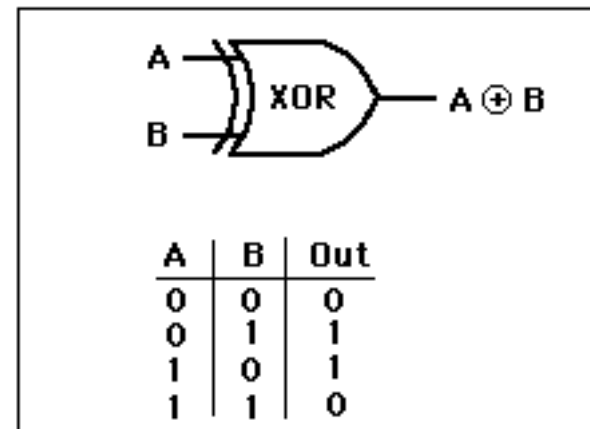
Avvolgere in caso di overflow

$$\begin{array}{r} 01010011 \\ + 01100110 \\ \hline 10111001 \end{array}$$

$$\begin{array}{r} 10111001 \\ + 01110100 \\ \hline 00101101 \end{array}$$



Il Complemento a uno = **11010001**



$$\begin{array}{r} 01010010 \\ + 01100111 \\ \hline 10111001 \end{array}$$

Per rilevare gli errori, il ricevitore aggiunge le quattro parole (le tre parole originali e il checksum). Se la somma contiene uno zero, il ricevente sa che c'è stato un errore. Verranno rilevati tutti gli errori a un bit, ma gli errori a due bit possono non essere rilevati (ad esempio, se l'ultima cifra della prima parola viene convertita in uno 0 e l'ultima cifra della seconda parola viene convertita in un 1).



Esercizi - Livello di Trasporto

P37: Paragonate Go-Back-N, Selective Repeat, e TCP (senza il delayed ACK). Assumete che i valori di timeout per tutti questi tre protocolli siano sufficientemente grandi per cui 5 segmenti di dati consecutivi e i corrispondenti ACK possano essere ricevuti (se non persi nel canale) dall'host ricevente (Host B) e dall'host mittente (Host A) rispettivamente. Supponete che l'Host A invii cinque segmenti di dati all'HOST B e che il secondo segmento (inviato da A) venga perso. Alla fine, tutti e 5 i segmenti di dati sono stati ricevuti correttamente dall'Host B.

- a. Quanti segmenti ha inviato l'Host A in tutto e quanti ACK ha mandato l'Host B in tutto? Quali sono i loro numeri di sequenza? Rispondete a queste domande per tutti e tre i protocolli.
- b. Se i valori di timeout per tutti e tre i protocolli sono molto più grandi di 5 RTT, quale protocollo consegnerà con successo tutti e cinque i segmenti di dati nell'intervallo di tempo più breve?



Soluzione Esercizio P37.a

- a. Quanti segmenti ha inviato l'Host A in tutto e quanti ACK ha mandato l'Host B in tutto? Quali sono i loro numeri di sequenza? Rispondete a queste domande per tutti e tre i protocolli.

Go-Back-N:

A invia 9 segmenti in totale. Vengono inizialmente inviati i segmenti 1, 2, 3, 4, 5 e successivamente i segmenti 2, 3, 4 e 5.

B invia 8 ACK. Sono 4 ACKS con numero di sequenza 1 ,e 4 ACKS con numeri di sequenza 2, 3, 4 e 5.

Selective Repeat:

A invia 6 segmenti in totale. Inizialmente vengono inviati i segmenti 1, 2, 3, 4, 5 e successivamente il segmento 2.

B invia 5 ACK. Sono 4 ACK con numero di sequenza 1, 3, 4, 5. E c'è un ACK con numero di sequenza 2.

TCP:

A invia 6 segmenti in totale. Inizialmente vengono inviati i segmenti 1, 2, 3, 4, 5 e successivamente il segmento 2.

B invia 5 ACK. Sono 4 ACK con numero di sequenza 2. C'è un ACK con numero di sequenza 6. **Si noti che TCP invia sempre un ACK con numero di sequenza previsto.**

